



**SCHOOL OF ENGINEERING, ENERGY AND THE BUILT
ENVIRONMENT**

**DEPARTMENT OF ENERGY AND ENVIRONMENTAL
ENGINEERING**

Horizontal Axis Wind Turbine DESIGN.

PRESENTED BY: Daniel Shauri

REG NO: BEET/ 2023/64362

**A RESEARCH PROJECT SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF A BACHELOR OF SCIENCE IN ENERGY AND
ENVIRONMENTAL TECHNOLOGY AT MOUNT KENYA UNIVERSITY**

DECLARATION

I hereby declare that this project report is based on my original work except for citations and quotations which have been duly acknowledged. I also declare that it has not been previously and concurrently submitted for any other diploma or award at Mount Kenya University.

Signature:  _____

Name: _____ DANIEL SHAURI _____

ID No.: _____ 37774051 _____

Date: _____ 18/09/2026 _____

SUPERVISOR

I the undersigned do hereby certify that this is a true report for the project undertaken by the above named student under my supervision and that it has been submitted to Mount Kenya University with my approval

Signature..... Date.....18th September 2026.....

ABSTRACT

The global transition toward sustainable energy necessitates efficient wind turbine design, yet traditional methodologies remain computationally expensive and manually intensive. This project addresses the significant accessibility gap in the industry caused by the high licensing costs of commercial platforms and the steep learning curves associated with command-line-based academic tools. The primary objective of this research was to develop "BladeCraft," an automated Python-based desktop application designed to streamline the aerodynamic optimization and geometric modeling of horizontal-axis wind turbines (HAWTs).

The methodology involved a multi-stage development process, beginning with the validation of core design logic in a Microsoft Excel prototype. This logic was subsequently translated into a standalone application using python programming language, featuring a core calculation engine rooted in Blade Element Momentum (BEM) theory. The system allows users to input key design parameters, such as tip speed ratio, wind speed, and lift coefficients, to generate optimized chord and twist distributions across between 5 to 1000 distinct blade sections.

To evaluate system performance, a 0.3m prototype utilizing the SG6043 airfoil was designed and simulated within the QBlade environment. The results demonstrated an aerodynamic power output of 26.7W at 1100 RPM and 7m/s wind speed, exceeding an initial 21.3W design target by 25.38%. After accounting for an 80% electromechanical efficiency factor, the predicted electrical output of 21.36W aligned almost exactly with the theoretical project requirements. This research provides a robust, manufacturing-ready bridge from theoretical optimization to physical realization through automated 3D modeling in Autodesk Fusion 360, offering an accessible solution for students and independent engineers.

Table of Contents

1. CHAPTER ONE.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
1.3 Objectives	2
1.3.1 Main Objective.....	2
1.3.2 Specific Objectives	2
1.4 Justification	3
1.4.1 Block Diagram.....	4
1.4.2 Explanation of blocks	4
2. CHAPTER TWO: LITERATURE REVIEW.....	6
2.1 Literature Review.....	6
2.1.1 Introduction.....	6
2.1.2 Theoretical Foundations of Wind Turbine Design	6
2.1.3 Existing Wind Turbine Design Technologies.....	9
2.1.4 The Research Gap and Project Contribution	11
3. CHAPTER THREE: SYSTEM DESIGN.....	14
3.1 Introduction.....	14
3.2 System Architecture.....	15
Model.....	15
View.....	16
Controller.....	16
3.3 Component Analysis.....	17
The User Interface (GUI) Module	17
Design & Implementation	17
3.4 The Core Calculation Engine.....	19
Input parameters	19
Output Parameters	20
Mathematical Derivations.....	21
Code and Iterative Structure Design.....	24
3.5 The Data Export Module.....	39
Function	39

Design & Implementation	40
3.6 System Performance and Analysis.....	41
Time Response	41
Sensitivity Analysis	41
4. CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING	52
4.1 Results and Discussion	52
Introduction.....	52
Prototype Design and Scaling.....	52
Power-Radius Relationship Implementation	52
Prototype Design Target:	53
Blade Geometry Generation via BladeCraft	53
Computational Performance Analysis (QBlade).....	55
Simulation Results	56
Discussion of results	56
4.2 Summary	57
5. CHAPTER FIVE: LIMITATIONS, CONCLUSIONS AND RECOMMENDATIONS	60
5.1 Limitations	60
5.2 Conclusion	60
5.3 Recommendations.....	61
References.....	62

List of Figures

1. Figure 2.1: *Illustration of wind turbine airfoil mechanics (Wind Turbine: How It Works, Parts, and Existing Types, 2025):* Page 9
2. Figure 3.1: *Block Diagram of the software process (Shauri, 2026):* Page 14
3. Figure 3.2: *Sensitivity of Rotor Dimensions on Wind Speed:* Page 45
4. Figure 3.3: *Sensitivity of Blade Dimensions on Tip Speed Ratio:* Page 52
5. Figure 4.1: *SG6043 Airfoil (SG6043 (Sg6043-il), n.d.):* Page 54
6. Figure 4.2: *Power Output Vs. Rotational Speed:* Page 57
7. Figure 4.4: *Autodesk Design Screenshot:* Page 60
8. Figure 4.5: *Blade construction automation script extract for Autodesk Fusion (Python)* Page 60

List of Tables

1. Table 3.1: *User Input Parameters:* Page 19
2. Table 3.2: *Fixed Input Parameters:* Page 20
3. Table 3.3: *Global Parameters:* Page 20
4. Table 3.4: *Sectional Parameters:* Page 21
5. Table 3.5: *Parameter Summary:* Page 42
6. Table 3.6: *Wind Speed Vs. Rotor Dimensions:* Page 43
7. Table 3.7: *Power Coefficient Vs. Rotor Dimensions:* Page 46
8. Table 3.8: *Tip Speed Ratio Vs. Rotor Dimensions:* Page 48
9. Table 4.1: *Prototype Blade Geometry Extract:* Page 56

List of Abbreviations

1. HAWT: Horizontal Axis Wind Turbine
2. AOA: Angle of Attack
3. DPU: Data Processing Unit
4. CFD: Computational Fluid Dynamics
5. BEM: Blade Element Momentum
6. CAD/CAM: Computer Aided Design/ Manufacturing
7. MVC: Model-View-Controller

1. CHAPTER ONE

1.1 Introduction

Wind energy has emerged as a crucial component of the global transition toward sustainable energy solutions. As the demand for renewable energy sources continues to rise, horizontal-axis wind turbines (HAWTs) play a pivotal role in harnessing wind power for electricity generation. However, the design and manufacturing of these turbines remain complex, requiring precise calculations, material selection, and optimization to ensure efficiency and reliability.

The traditional approach to wind turbine design involves manual calculations and extensive engineering expertise, which can be time-consuming and prone to errors. Engineers must account for various parameters, including air velocity, tip speed ratio, lift coefficient, and angle of attack, to optimize turbine performance. This complexity often leads to inefficiencies in the design process, limiting accessibility for a wider range of manufacturers and independent engineers.

To address these challenges, this project proposes the development of a professional-grade automatic wind turbine design system. I will begin by creating a functional prototype, which will be in the form of an automatic wind turbine design spreadsheet in Microsoft Excel. I will use this to validate the core design logic and calculations for HAWTs.

Building on this foundation, the primary objective is to translate the proven Excel model into a dedicated Python-based desktop application. This new approach enhances ease of use, reduces design errors, and improves efficiency by providing a more robust, scalable, and user-friendly alternative to both manual methods and complex, expensive software. By leveraging the power of a custom-built application, this system seeks to bridge the gap in accessibility, making advanced wind turbine design more achievable for engineers and manufacturers alike.

1.2 Problem Statement

Despite advancements in wind energy technology, the design and optimization of wind turbines remain a significant challenge. Traditional methods require extensive expertise and computational resources, making the process inaccessible to many engineers and

manufacturers. Additionally, manual calculations are susceptible to errors, leading to suboptimal turbine performance and increased production costs.

Research indicates that wind turbine efficiency is heavily dependent on precise aerodynamic calculations and material selection. However, existing design methods often fail to integrate automation, resulting in inefficiencies and prolonged development timelines. The lack of a streamlined, user-friendly design system hinders the broader adoption of wind energy technology.

1.3 Objectives

1.3.1 Main Objective

To build an automatic wind turbine design system based on Microsoft Excel that enhances ease and efficiency in designing and manufacturing suitable wind turbines, ensuring accurate calculations, optimal performance, and streamlined production processes.

1.3.2 Specific Objectives

- a. Investigate existing wind turbine design methodologies and identify challenges associated with manual calculations and optimization.
- b. Develop an automated computational framework starting with Microsoft Excel then using this as a foundation for a fully-fledged desktop application based on python for wind turbine design
- c. Design a user-friendly interface that enables engineers and manufacturers to input relevant parameters and receive optimized design specifications.
- d. Analyze the performance and accuracy of the developed system by comparing computed results with conventional wind turbine design methods.
- e. Implement error handling mechanisms to validate input parameters and prevent incorrect design outputs.
- f. Optimize the wind turbine blade shape dynamically based on input data, providing real-time visualization and performance analysis.
- g. Evaluate the efficiency and reliability of the system through experimental

testing and performance validation against industry standards.

1.4 Justification

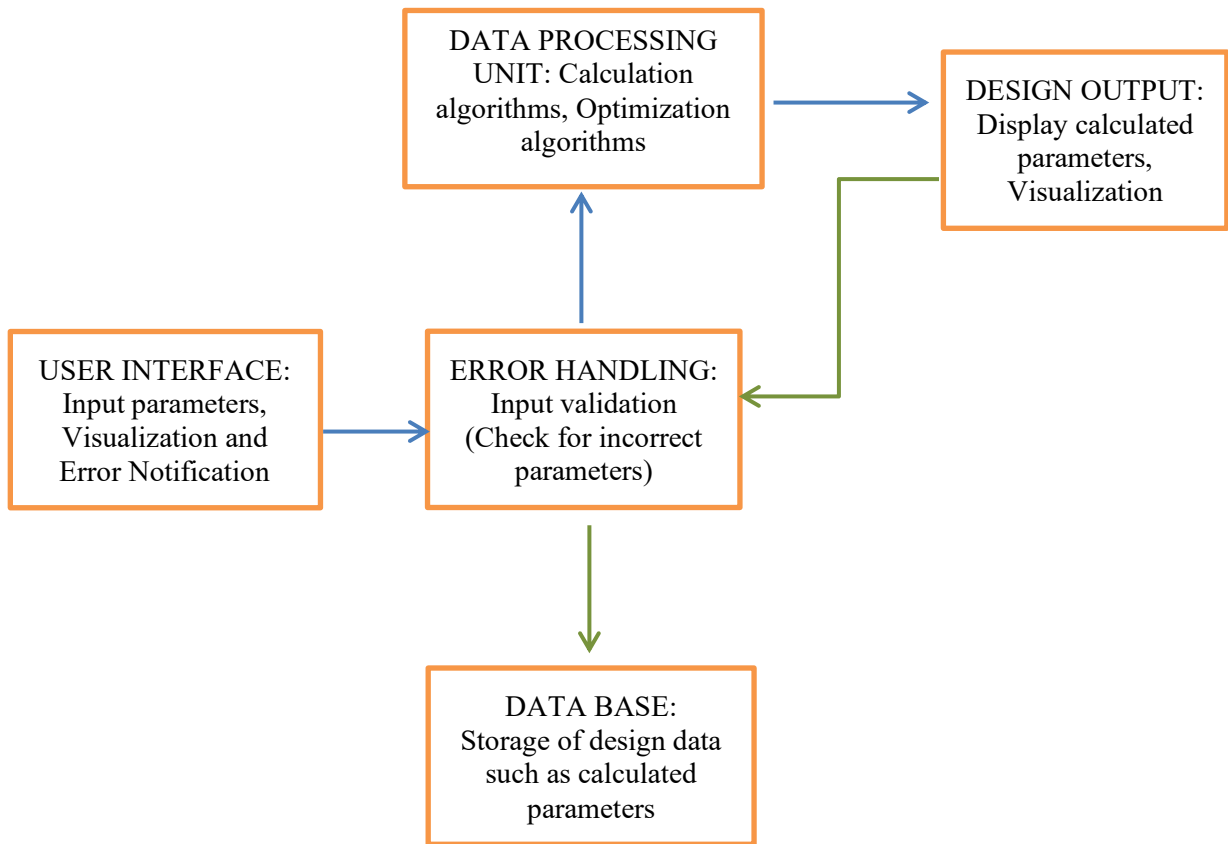
The proposed automatic wind turbine design system represents a forward-thinking and timely response to the growing demand for accessible and efficient renewable energy solutions. Traditional design platforms like MATLAB and ANSYS are powerful but costly and require specialized knowledge, often making them out of reach for small manufacturers or independent engineers.

This project simplifies the design process by first using a Microsoft Excel spreadsheet as a functional prototype, then transitioning to a dedicated Python-based application. This approach transforms an otherwise complex and exclusive task into a more user-friendly and inclusive solution.

By leveraging a custom-built desktop application, this system introduces automation into a traditionally manual and technical process, significantly reducing human error and saving time. It enables real-time calculations and dynamic blade optimization, allowing users to visualize and refine their turbine designs instantly based on variable inputs. This not only enhances the accuracy of the final output but also empowers a wider range of users to engage in wind energy development without needing advanced engineering software.

Furthermore, the project aligns seamlessly with the global movement toward clean energy. As investments in wind power grow, the need for practical, scalable tools becomes more pressing. This Python-based system answers that call by offering a cost-effective, efficient, and adaptable design framework. It provides a solid foundation for future enhancements and supports the widespread adoption of sustainable energy practices through innovation that is both technically sound and accessible.

1.4.1 Block Diagram



1.4.2 Explanation of blocks

a. User-Interface

The user interface component serves as the front-end of the system, allowing users to input various parameters such as air velocity, tip speed ratio, lift coefficient, angle of attack, number of sections, and required power. It also includes a dynamic visualization feature that updates the wind turbine blade shape based on the input data. Additionally, it provides error notifications to alert users when incorrect parameters are entered, ensuring accurate and efficient design processes.

b. Data-Processing-Unit

This is the core computational engine of the system, responsible for executing complex calculations and optimization algorithms. It processes the input parameters to compute key

design elements such as rotor area, radius, and chord length. The optimization algorithms ensure that the wind turbine design achieves optimal performance and efficiency, taking account of the specific site requirements and user inputs.

c. **Output**

Design output is the component that handles the presentation of the computed results and the dynamic visualization of the wind turbine blade shape. It displays the calculated parameters such as rotor area, radius, and chord length, providing users with a clear and concise summary of the design. The visualization feature helps users understand how different input parameters affect the overall shape and performance of the wind turbine.

d. **Database**

The database stores all user inputs, calculated parameters, and design outputs for future reference. It enables users to save their design data and retrieve it later for further analysis or modifications. By maintaining a comprehensive record of all design activities, the database ensures that users can track their progress and make informed decisions based on historical data.

e. **Error-handling**

Error handling is the component responsible for validating the input parameters and ensuring that only correct and feasible values are processed by the system. It checks for errors such as out-of-range values or incompatible parameter combinations and triggers error notifications when necessary. This helps prevent incorrect designs and ensures that the final wind turbine design meets the required standards of accuracy and reliability.

2. CHAPTER TWO: LITERATURE REVIEW

2.1 Literature Review

2.1.1 Introduction

The objective of this literature review is to survey, critically evaluate, and synthesize existing research and technologies related to wind turbine design, identifying a critical gap that the project aims to fill. This analysis will serve to justify the relevance and unique contribution of our proposed solution, positioning it as a novel tool that addresses the current limitations of both expensive commercial software and manual academic approaches.

To achieve this, the review will first establish the fundamental theoretical principles of wind turbine design, with a specific focus on the aerodynamics of horizontal-axis wind turbines (HAWTs) and key design parameters such as the tip speed ratio and lift coefficient (*How Efficient Are Wind Turbines in 2026? Explained*, n.d.). Following this, it will provide a critical analysis of existing software platforms, including high-cost commercial tools like MATLAB and ANSYS, as well as manual, open-source academic methods, evaluating their respective strengths and weaknesses. By doing so, the review will demonstrate a significant gap in the market for a cost-effective, user-friendly, and highly automated desktop application that streamlines the HAWT design process, a void that the project is specifically designed to fill.

2.1.2 Theoretical Foundations of Wind Turbine Design

The Blade Element Momentum (BEM) theory stands as the most widely adopted method for the aerodynamic analysis and design of wind turbine blades. Developed in a comprehensive form by Hermann Glauert in the 1930s, BEM theory combines two fundamental aerodynamic principles: blade element theory and momentum theory. The blade element theory divides the turbine blade into a number of independent radial sections or "blade elements" (Duran et al., 2020). For each element, the aerodynamic forces (lift and drag) are calculated using two-dimensional airfoil data, as if each section were an isolated wing. Simultaneously, momentum theory analyzes the flow of air through an imagined "actuator disk" that represents the turbine rotor, accounting for the change in air velocity as it passes through (Ishak et al., 2022). By iteratively combining these two theories

(matching the forces predicted by blade element theory with the changes in momentum predicted by momentum theory) the BEM method can accurately predict the aerodynamic performance of the entire rotor.

BEM theory calculates the optimal chord length along the blade, which is critical for achieving maximum aerodynamic efficiency. The chord length, denoted by c , for a specific blade section at a radius r is often calculated using the following formula, derived from BEM principles (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$c(r) = \frac{8\pi r}{N C_L} \left(\frac{1 - \cos \phi}{2} \right)$$

Where:

- a. $c(r)$: The chord length of the blade at a given radius r from the hub.
- b. r : The radial distance from the hub to the specific blade element being analyzed.
- c. N : The number of blades on the turbine. This highlights how the total lift is distributed across the blades.
- d. C_L : The lift coefficient, a dimensionless quantity that relates the lift generated by the airfoil to the fluid density, flow velocity, and a reference area. This value is determined from airfoil data at a specific angle of attack.
- e. ϕ : The flow angle, which is the angle between the relative wind velocity and the plane of rotation. It is a crucial parameter that changes with the radial position along the blade.
- f. This mathematical framework makes BEM a powerful tool, as it simplifies the complex three-dimensional flow field into a series of manageable two-dimensional calculations, making it computationally efficient and a cornerstone of modern wind turbine blade design.

The key design parameters in the design of a HAWT using the BEM method are:

- a. Design Wind Speed (V_D): The Designed Wind Speed is the specific wind velocity at which the turbine is expected to operate at its maximum efficiency. For my design, this value is 4 m/s. This parameter is crucial because all other design variables, such as blade geometry and rotor size, are optimized to capture the maximum possible power from the wind at this specific speed.
- b. Required Power Output (P_T): The Required Power Output is the target electrical power the turbine is intended to generate. My design specifies a power output of 100 W. This value directly influences the size and geometry of the entire turbine system, including the rotor radius and the required aerodynamic forces on the blades.
- c. Tip Speed Ratio (λ): The Tip Speed Ratio (TSR), denoted as λ , is a critical dimensionless parameter that describes the relationship between the tangential speed of the blade tip and the wind speed. My design specifies a TSR of 5, which is a common value for modern, high-speed wind turbines. The TSR is crucial because it directly influences the aerodynamic efficiency of the turbine. A higher TSR generally allows the turbine to operate more efficiently at lower wind speeds, but it can also increase noise and structural stress. Choosing an optimal TSR is a key design trade-off to maximize the power coefficient while minimizing loads.
- d. Lift (C_L) and Drag (C_D) Coefficients: Lift and Drag Coefficients are fundamental to understanding the forces acting on the wind turbine blade. The lift coefficient is a measure of the upward force generated by the blade's airfoil, which is the primary force that drives the rotor's rotation. The drag coefficient represents the resistance force that opposes the blade's motion. The goal of any blade design is to maximize the lift-to-drag ratio, as this ensures that the blade generates as much rotational force as possible with minimal wasted energy due to drag. My design has a C_L of 0.8.
- e. Angle of attack (α): The Angle of Attack (AOA), denoted as α , is the angle between

the relative wind velocity and the blade's chord line. My design sets a specific AOA of 4 degrees. This is one of the most critical variables for optimizing lift. Each airfoil has a characteristic performance curve that plots its lift and drag coefficients against various angles of attack. By selecting an optimal AOA, such as 4 degrees the blade is positioned to generate maximum lift and, consequently, maximum torque to spin the rotor. If the angle of attack is too low, the blade will generate insufficient lift; if it is too high, the airfoil will "stall," causing a dramatic drop in lift and a sharp increase in drag, which can halt the turbine's rotation.

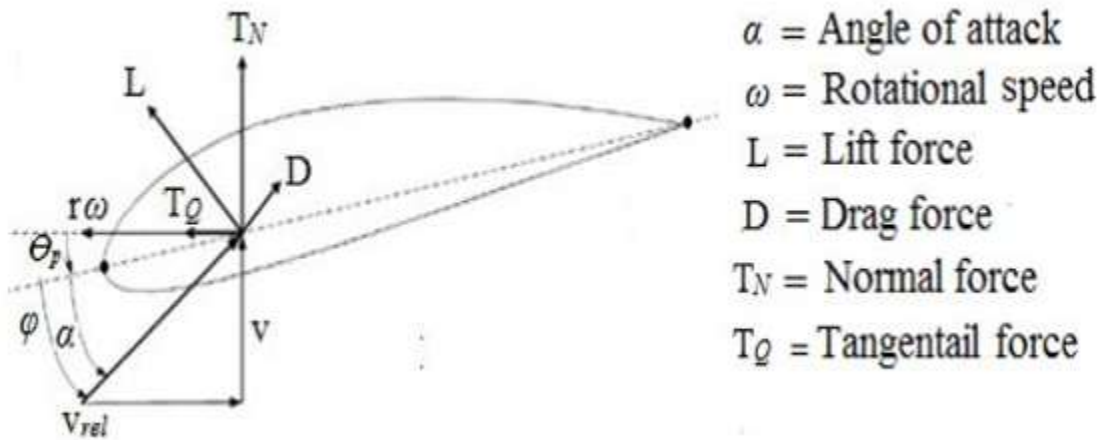


Figure 2.1: Illustration of wind turbine airfoil mechanics (*Wind Turbine: How It Works, Parts, and Existing Types*, 2025)

2.1.3 Existing Wind Turbine Design Technologies

As the global transition toward sustainable energy sources gains momentum, wind power stands out as a leading contender due to its maturity and increasing cost-effectiveness. However, a comprehensive analysis of existing wind turbine design technologies reveals that while they offer significant advantages, they are also constrained by notable limitations. This section will critically examine the current landscape of design systems and scrutinize key design and operational aspects using both high-end commercial software and open-source academic methodologies (Cope, 2025). By highlighting both the technological advancements that have propelled the industry forward and the persistent challenges that impede further growth, this analysis aims to provide a nuanced understanding of the strengths and weaknesses inherent in today's wind power solutions, paving

the way for future innovation.

a. High-End Commercial Software

High-end commercial software packages such as MATLAB, ANSYS, and SolidWorks are the bedrock of advanced engineering and design in the wind energy industry. These platforms are used for a wide range of sophisticated applications, including detailed Computational Fluid Dynamics (CFD) analysis to simulate airflow over intricate blade geometries (ANSYS), comprehensive finite element analysis (FEA) to evaluate structural integrity under dynamic loads (ANSYS, SolidWorks Simulation), and complex mathematical modeling and algorithm development for turbine control systems (MATLAB). The primary strength of these platforms lies in their power, reliability, and robustness. As industry-standard tools, they offer a vast array of validated features and extensive libraries, ensuring that the results of simulations and designs are highly accurate and trusted for large-scale, mission-critical projects. For major corporations and research institutions, this level of precision and proven capability is indispensable (Kusiak & Zhang, 2010).

However, despite their power, these commercial tools present significant limitations, particularly in the context of broader accessibility and innovation. The most immediate weakness is their exorbitant cost, with licensing fees that can range from thousands to tens of thousands of dollars annually, effectively creating a prohibitive barrier to entry for small firms, startups, and individual engineers (Wood, 2011). Furthermore, the immense complexity of these platforms necessitates a steep learning curve and often requires specialized, costly training to master their full capabilities (*Economics*, 2026). This combination of high financial investment and extensive time commitment makes them fundamentally inaccessible to a wider community of innovators, stifling the democratized development and exploration of new wind turbine technologies outside of established, well-funded organizations (Eg, 2024).

b. Open-source and Academic Methodologies.

In contrast to commercial software, open-source and academic methodologies provide a flexible and cost-effective alternative for researchers and engineers. These tools often take the form of specialized programming libraries (e.g., Python libraries for data analysis and numerical computation) or dedicated open-source codes (e.g., OpenFOAM for CFD analysis) (Jannet et al., 2020). The primary advantage of these platforms is their accessibility; they are typically free to use and distribute, which eliminates the financial barrier associated with commercial licensing. Moreover, their open nature allows for unparalleled flexibility. Researchers can directly access and modify the underlying source code to develop custom algorithms, integrate new theoretical models, or tailor the analysis to highly specific, unconventional problems, fostering rapid innovation within the academic community (*What Materials Are Used to Make Wind Turbines?*, 2018).

However, the very features that make these tools powerful for a niche audience, also present critical drawbacks that hinder their widespread adoption. A major limitation is the absence of a user-friendly graphical interface, as these tools are often operated through command-line inputs or custom-coded scripts (Sørensen & Schreck, 2009). This lack of a streamlined user experience creates a steep barrier for anyone without a strong background in programming and computational methods, making them unsuitable for a streamlined, commercial design workflow. Additionally, the decentralized nature of open-source development means there is no dedicated customer support or standardized, comprehensive documentation (*Wind Turbine Recycling*, 2026). Users are typically reliant on community forums or their own problem-solving skills, which can make troubleshooting complex issues time-consuming and inefficient. Consequently, while ideal for focused research, these tools lack the usability and robust support required for broad industry application.

2.1.4 The Research Gap and Project Contribution

The critical examination of existing wind turbine design software reveals a significant and unresolved research gap. While high-end commercial platforms like ANSYS and MATLAB offer unparalleled computational power and validation for complex simulations, their high cost and steep learning curve make them largely inaccessible to a wider user base

of students, small firms, and independent engineers. Conversely, open-source and academic methodologies, though free and highly flexible, often lack a user-friendly interface and comprehensive support, making them unsuitable for a streamlined design workflow (Science Insights Team, 2026a). The literature, therefore, highlights a critical need for a tool that can bridge this divide, one that combines robust computational capability with the accessibility, affordability, and intuitive design required for a non-specialist audience.

This project is a direct response to this identified gap. By taking the proven calculation logic that was successfully prototyped in a spreadsheet application, this work will package it into a self-contained, user-friendly Python application. The program name will be “BladeCraft”. This is a fusion of two key concepts central to the project:

- Blade: This refers to the core component of a wind turbine that you are designing and optimizing. It’s the essential element that harnesses the wind's power.
- Craft: This suggests skill, artistry, and precision. It conveys the idea that the program is not just a calculator but a tool that helps users meticulously craft a functional and efficient wind turbine blade.

The name is memorable, concise, and immediately communicates the purpose of the software: to expertly design and create the most critical part of a wind turbine. It encapsulates both the technical and creative aspects of the engineering process. BladeCraft will provide an intuitive graphical interface that simplifies the input of design parameters and visualizes the results of complex aerodynamic calculations. This approach directly addresses the limitations of both commercial and open-source tools by delivering a powerful yet accessible design utility that lowers the barrier to entry for aspiring wind energy engineers and eases the initial design phase of horizontal-axis wind turbines (Wanderley & Levi, 2023).

My work stands out because of the following:

- **Accessibility:** BladeCraft offers a cost-effective alternative to expensive commercial software, making advanced design tools available to students, small firms, and independent engineers.
- **Automation:** The application automates the traditionally manual and repetitive calculations involved in wind turbine blade design, significantly reducing the potential for human error.
- **User-Centric Design:** Unlike many command-line-based academic tools, BladeCraft prioritizes a simple, intuitive graphical user interface (GUI) that streamlines the design workflow.
- **Visualization:** It provides dynamic, real-time visualization of the blade geometry and aerodynamic properties, which is a feature often lacking in more rudimentary design tools.
- **Manufacturing Ready Exports:** The final designs can be exported to standard formats like CSV for data analysis or directly to formats compatible with CAD/CAM software and 3D printing, facilitating the transition from design to manufacturing.

3. CHAPTER THREE: SYSTEM DESIGN

3.1 Introduction

This chapter provides a detailed description of the BladeCraft application's system design, structured according to the functional blocks outlined in the system block diagram. The analysis begins with the User Interface, which serves as the primary point of interaction for data input and the display of results. User inputs are then passed to an Error Handling module that validates the parameters before they are processed. Validated data is then sent to the core of the system, the Data Processing Unit, which contains the calculation and optimization algorithms derived from Blade Element Momentum theory. The outputs from this unit are then passed to the Design Output module for display and visualization, and also to a Data Base for storage. Each of these system parts will be analyzed in detail to describe their specific design and functions, including the mathematical proofs that govern the system's logic and performance.

Block Diagram

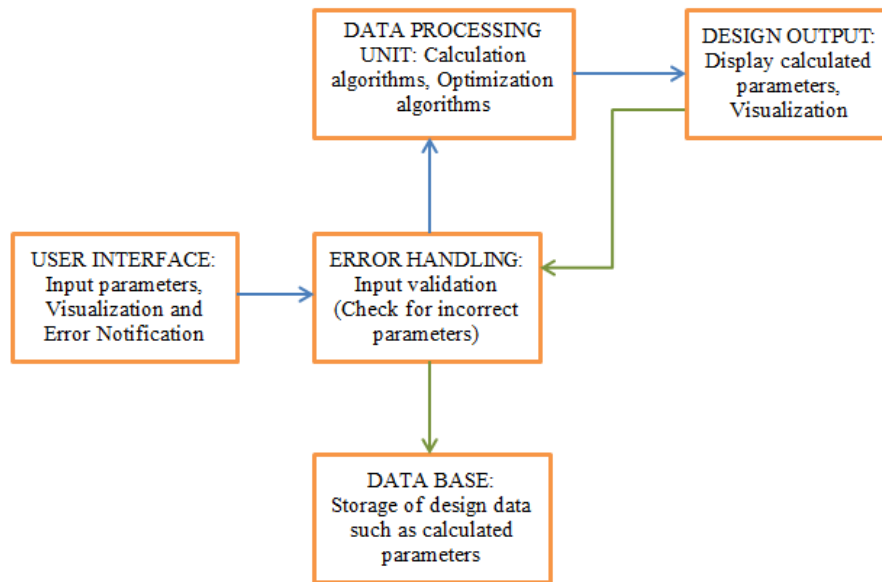


Figure 3.1: Block Diagram of the software process (Shauri, 2026)

The block diagram illustrates the system architecture and data flow of the BladeCraft application. The process begins at the User Interface, where the user inputs design parameters. This data is

immediately passed to an Error Handling module, which validates the inputs to ensure they are within acceptable ranges. If the parameters are correct, they are sent to the core Data Processing Unit, where calculation and optimization algorithms are executed to determine the blade geometry. The results are then sent to the Design Output module to be displayed to the user through tables and visualizations. Concurrently, the calculated parameters are also sent to a Data Base for storage and later retrieval, such as for exporting to a file. Any errors detected during validation are communicated back to the User Interface for notification.

3.2 System Architecture.

The overall architecture of the BladeCraft application can be understood using the Model-View-Controller (MVC) design pattern. MVC is a widely used software architectural pattern for implementing user interfaces. Its primary purpose is to separate the application's concerns into three distinct, interconnected components. This separation makes the code more organized, easier to maintain, and simpler to update, as changes to one component (like the user interface) have minimal impact on the others (like the core calculations).

Model

The Model is the central component of the pattern and acts as the "brain" of the application. It is responsible for managing the application's data, logic, and rules. The Model contains the core functionality and is completely independent of the user interface. It receives input from the Controller, processes it according to its internal logic, and makes the results available.

In the BladeCraft application, the Model is implemented as the core calculation engine, specifically the `calculate_blade_design` function. This function contains all the fundamental aerodynamic and engineering formulas from Blade Element Momentum theory. It accepts a set of numerical parameters, performs the iterative calculations to determine the blade's geometry and performance characteristics, and returns the results in a structured data format (a Python dictionary), without any knowledge of how this data will be displayed.

View

The View is the component responsible for presenting the data from the Model to the user. It is what the user sees and interacts with. The View's job is to render the user interface, display data, and pass user actions (like button clicks or text input) to the Controller. It does not contain any application logic.

In the BladeCraft application, the View is implemented as the user interface (GUI), which is defined within the MainWindow class. This includes all the visual elements created with the PyQt5 library, such as the main window, the tabbed interface (QTabWidget), the input fields (QLineEdit), the data tables (QTableWidget), and the Matplotlib canvas used for the blade visualization. Its sole purpose is to display the input parameters and the calculated results to the user.

Controller

The Controller acts as the intermediary between the Model and the View. It listens for user input from the View, processes it, and then tells the Model what to do. After the Model has updated its data, the Controller is responsible for telling the View to refresh itself to display the new data.

In the BladeCraft application, the Controller is implemented as the event handlers in the GUI code. The most prominent example is the `run_calculations` function. When the user clicks the "Calculate Design" button, this function is triggered. It acts as the Controller by first gathering the input data from the View (reading the text from the QLineEdit widgets). It then calls the Model (`calculate_blade_design`) to perform the core calculations. Finally, upon receiving the results from the Model, it updates the View by populating the tables and redrawing the visualization graph.

3.3 Component Analysis

The User Interface (GUI) Module

The User Interface (GUI) module serves as the primary interactive component of the BladeCraft system, designed to provide a clear and intuitive workflow for the end-user. Its main function is to facilitate the input of all necessary design parameters, trigger the core calculation engine, and present the complex output data in an easily digestible format through both tables and graphs. The entire interface was implemented using the Python programming language and the robust PyQt5 framework. The main window ('QMainWindow') is structured around a tabbed widget ('QTabWidget') that logically separates the application's functions into four distinct sections: a "Design Parameters" tab for user input, "Blade Geometry" and "Summary" tabs for detailed tabular results, and a "Visualization" tab for a graphical plot of the blade's profile.

Design & Implementation

The Graphical User Interface (GUI) was designed to be both functional and aesthetically pleasing, providing a clear and modern user experience. The entire GUI was developed using Python, chosen for its rapid development capabilities, and the PyQt5 library, a comprehensive and mature framework for creating professional desktop applications. The styling of the application is controlled dynamically using PyQt5's version of Cascading Style Sheets (QSS), allowing for features like theme switching and interactive element highlighting.

The main window of the application is a QMainWindow object, which serves as the primary container for all other widgets. To organize the different functions of the application logically, a QTabWidget is used as the central widget. This creates a tabbed interface that separates the workflow into four distinct sections, with the currently selected tab highlighted in yellow for clear visual feedback, a style controlled by the QTabBar::tab:selected QSS rule. The four tabs are detailed as follows:

- **Design Parameters Tab:** This is the primary input screen. It uses a QFormLayout to neatly arrange QLabel and QLineEdit widgets, allowing the user to enter all the necessary parameters for the blade design. At the bottom of this tab is the main

"Calculate Design" QPushButton. When this button is clicked, it initiates a short visual sequence to provide feedback to the user. A QProgressBar appears and fills over two seconds. Once complete, the progress bar vanishes and is replaced by a green "Completed ✓" message, confirming that the calculations have finished. This tab also contains a button to toggle the application's appearance between a dark mode and a light mode. This is achieved by calling a function that uses `setStyleSheet` to apply a completely new set of QSS rules to the entire application, changing the background, text, and widget colors instantly.

- **Blade Geometry Tab:** This tab is dedicated to displaying the detailed, sectional data of the generated blade. A `QTableWidget` is used to present the data in a clean, tabular format, with columns for sectional radius, local TSR, flow angle, blade setting angle, and chord length.
- **Summary Tab:** This tab provides a high-level overview of the most important inputs and calculated results. Similar to the geometry tab, it uses a `QTableWidget` to display key parameters like Rotor Radius, Rotor Area, and Solidity in a simple "Parameter" and "Value" format.
- **Visualization Tab:** To provide a visual representation of the designed blade, this tab integrates a Matplotlib plot directly into the PyQt5 interface using the `FigureCanvasQTAgg` widget. It displays a 2D plot of the blade's chord length versus its radius, effectively showing the blade's planform shape. A `NavigationToolbar` is included at the bottom of the plot, allowing the user to zoom, pan, and save the visualization.

Across all tabs, key interactive elements like the "Calculate," "Dark/Light," and "Export" buttons are styled to provide visual feedback. Using the `:hover` pseudo-state in the QSS stylesheet, these buttons change color to a distinct green when the user's mouse cursor hovers over them, indicating that they are clickable and improving the overall user experience.

3.4 The Core Calculation Engine

The Core Calculation Engine module functions as the central processing unit of the BladeCraft application, containing the software implementation of the established Blade Element Momentum (BEM) theory. Its primary purpose is to take the validated design parameters from the user interface and apply them to a series of aerodynamic and kinematic formulas to determine the optimal geometry for a high-efficiency wind turbine blade. This engine is responsible for all the critical calculations, including the ideal rotor radius, the chord length distribution, and the necessary blade twist at multiple sections along the blade's span. In essence, this module serves as the 'brains' of the system, transforming theoretical user requirements into a tangible and data-driven blade design ready for visualization and further analysis.

Input parameters

➤ User Input Parameters

User input parameters are parameters that are defined by the user. They are summarized in Table 3.1

Table 3.1: User Input Parameters

Parameter Name	Symbol	Units
Required Power	P	Watts (W)
Design Wind Speed	V	Meters per second (m/s)
Tip Speed Ratio	λ	Dimensionless
Power Coefficient	C _p	Dimensionless
Efficiency	η	Dimensionless
Lift Coefficient	C _L	Dimensionless
Angle of Attack	α	Degrees (°)
Number of Sections	n	Integer

➤ Fixed Input Parameters

Fixed input parameters (Constants) are constants hard-coded into the application that are used in the calculations but are not changed by the user on the main input screen. They are summarized in Table 3.2

Table 3.2: Fixed Input Parameters

Parameter Name	Symbol	Units	Value
Air Density	ρ	Kilograms per cubic meter (kg/m ³)	1.224
Number of Blades	N	Integer	3

Output Parameters

These parameters are divided into two categories: Global Parameters that describe the entire rotor, and Sectional Parameters that are calculated for each specific segment along the blade's length.

➤ Global Parameters

Global Parameters are those which describe the entire rotor. They are summarized in Table 3.3

Table 3.3: Global Parameters

Parameter Name	Symbol	Units
Rotor Radius	R	Meters (m)
Rotor Area	A	Square Meters (m ²)
Single Blade Area	AB	Square Meters (m ²)
Total Blade Area	AT	Square Meters (m ²)
Solidity	σ	Dimensionless

➤ **Sectional Parameters**

These are calculated iteratively for each section at a local radius, r . They are summarized in Table 3.4

Table 3.4: Sectional Parameters

Parameter Name	Symbol	Units
Local Tip Speed Ratio	λr	Dimensionless
Flow Angle	Φ	Degrees (°)
Blade Setting Angle (Twist)	β	Degrees (°)
Chord Length	c	Meters (m)

Mathematical Derivations

The Core Calculation Engine operates based on a set of established formulas derived from Blade Element Momentum (BEM) theory. These equations govern the relationship between the wind's power, the turbine's dimensions, and the aerodynamic forces acting on the blades. The following derivations represent the key mathematical proves implemented in the software to calculate the optimal blade geometry.

1. Rotor Radius (R)

The first and most fundamental calculation determines the overall size of the turbine rotor required to capture the target power from the wind. This is derived from the basic wind power equation (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

Symbol: R

Units: Meters (m)

Formula:

$$R = \sqrt{\frac{2P}{\pi\rho V^3 C_p \eta}}$$

This formula calculates the Rotor Radius (R) needed to achieve the user's Required Power

(P) from a given Design Wind Speed (V). It accounts for the Air Density (ρ), the Power Coefficient (C_p) which represents the rotor's aerodynamic efficiency, and the overall mechanical Efficiency (η) of the system. The strong dependence on the cube of the wind speed (V^3) highlights how critical the wind resource is to the turbine's size

2. Local Tip Speed Ratio

The Tip Speed Ratio (λ) is the ratio of the blade tip's speed to the wind speed. To design the blade's twist correctly, we must know this ratio at every point along the blade's length (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

Symbol: λ_r

Units: Dimensionless

Formula:

$$\lambda_r = \frac{\lambda r}{R}$$

This formula calculates the Local Tip Speed Ratio (λ_r) at a specific sectional radius (r) along the blade. It scales the overall Tip Speed Ratio (λ) by the section's fractional distance from the center of the rotor (r/R). This value is crucial because it determines the angle of the relative wind at each section.

3. Flow Angle

The Flow Angle is the angle of the "relative wind", which is the combination of the natural wind and the wind created by the blade's rotation, as it is experienced by a specific blade section (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

Symbol: Φ

Units: Degrees ($^\circ$)

Formula:

$$\phi = \frac{2}{3} \tan^{-1} \left(\frac{1}{\lambda_r} \right)$$

This formula, a well-established approximation from BEM theory, calculates the Flow

Angle (ϕ) based on the Local Tip Speed Ratio (λ_r). It accurately represents the direction of the incoming wind that the airfoil section must be oriented to face. This angle is largest at the root of the blade (where λ_r is low) and smallest at the tip (where λ_r is high)

4. Chord Length

The Chord Length is the width of the blade at a specific section. Its size is optimized to generate the maximum amount of lift without stalling (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

Symbol: c

Units: Meters (m)

Formula:

$$c = \frac{8\pi r}{N C_L} \left(\frac{1 - \cos \phi}{2} \right)$$

This formula calculates the optimal Chord Length (c) for a blade section at radius (r). It is designed to ensure that the blade generates the necessary aerodynamic force based on the Number of Blades (N), the airfoil's Lift Coefficient (C_L), and the local Flow Angle (ϕ). This calculation results in a blade that is wide at the root, and which tapers to a narrow tip, a characteristic of modern, efficient wind turbine blades.

5. Blade Setting Angle (Twist)

Symbol: β

Units: Degrees ($^\circ$)

Formula (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$\beta = \phi - \alpha$$

6. Rotor Area

Symbol: A

Units: Square Meters (m^2)

Formula (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$A = \pi R^2$$

7. Single Blade Area

Symbol: A_B

Units: Square Meters (m^2)

Formula: The single blade area is the sum of all the individual trapezoidal segments of the blade from the root to the tip (Shauri, 2026). This is represented mathematically by the summation:

$$A_B = \sum_{i=1}^{n-1} \frac{1}{2} (c_i + c_{i+1}) \Delta r$$

8. Total Blade Area

Symbol: A_T

Units: Square Meters (m^2)

Formula (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$A_T = N A_B$$

9. Solidity

It measures the portion of the rotor area that is covered in blades.

Symbol: σ

Units: Dimensionless

Formula (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$\sigma = \frac{A_T}{A}$$

Code and Iterative Structure Design

While the mathematical formulas provide the means to calculate the properties of a single blade section, a complete blade is a complex, continuous 3D shape where these properties change along its length. To construct this shape, the Core Calculation Engine employs an iterative design

process. The software first discretizes the total rotor radius into a user-defined number of segments, creating a series of calculation points from the root to the tip. The code then initiates a loop, and for each radial step in this loop, it applies the full set of derived formulas for Local Tip Speed Ratio, Flow Angle, and Chord Length. By repeating these calculations for every section, the engine systematically builds a complete dataset of geometric points. This collection of points effectively defines the blade's continuous taper (the change in width) and twist (the change in angle), creating the full, optimized aerodynamic profile.

1. Rotor Radius

➤ Python Code

```
try:
    rotor_radius = math.sqrt(
        (2 * required_power) /
        (math.pi * AIR_DENSITY * power_coefficient * efficiency * (design_wind_speed ** 3))
    )
except (ValueError, ZeroDivisionError) as e:
    QMessageBox.warning(None, "Calculation Error", f"Cannot calculate rotor radius. Please check your inputs. Error: {e}")
    return None
```

This block of code is a direct implementation of the rearranged wind power formula, designed to solve for the Rotor Radius (R). It calculates the size of the rotor needed to generate the required_power given the available wind energy and the system's efficiencies.

- `math.sqrt(...)`: This is the function from Python's math library that calculates the square root of the entire expression contained within the parentheses.
- The Numerator (`2 * required_power`): This corresponds to the 2P term in the mathematical formula. It represents the target power output from the user.
- The Denominator (`math.pi * AIR_DENSITY * ...`): This part of the code calculates the total power available in the wind per unit of rotor area, adjusted for all system inefficiencies.
 - `math.pi`: The constant Pi (π).
 - `AIR_DENSITY`: The fixed constant for the density of air (ρ).
 - `power_coefficient`: The user-defined aerodynamic efficiency of the rotor (C_p).
 - `efficiency`: The user-defined mechanical and electrical efficiency of the

system (η).

- (design_wind_speed ** 3): The user-defined wind speed (V), raised to the power of 3. This reflects the fact that the power available in the wind is proportional to the cube of its speed.
- Error Handling (try...except): The entire calculation is wrapped in a try...except block. This is a crucial safety feature. It "tries" to perform the calculation, but if the user provides invalid inputs that would cause a mathematical error (like a design_wind_speed of zero, leading to a ZeroDivisionError), the program will not crash. Instead, it will "catch" the error, display a warning message to the user, and safely stop the calculation process.

➤ Excel Formula

`=SQRT((2*C7)/(PI()*C8*C12*C11*(C9^3)))`

This formula is designed to calculate the Rotor Radius (R) by implementing the rearranged wind power equation. It takes several input values, each stored in a specific cell, to compute the result.

- =SQRT(...): This is the main Excel function that calculates the square root of the entire expression contained within the parentheses. It is the final step in the calculation.
- The Numerator (2*C7): This part corresponds to the 2P term in the mathematical formula.
 - C7 is the cell that contains the value for the Required Power (P) in Watts.
- The Denominator (PI()*C8*C12*C11*(C9^3)): This section of the formula calculates the total power available in the wind per unit of area, adjusted for all system inefficiencies. It is a multiplication of several terms:
 - PI(): This is Excel's built-in function for the constant Pi (π).
 - C8: This cell contains the value for the Air Density (ρ).
 - C12: This cell contains the value for the Power Coefficient (C_p).
 - C11: This cell contains the value for the overall system Efficiency (η).
 - (C9^3): This takes the value from cell C9, which is the Design Wind Speed

(V), and raises it to the power of 3 using the ^ operator.

The formula takes all the necessary inputs from their respective cells and calculates the rotor radius in a single, efficient step, exactly like the Python code but within the spreadsheet environment.

2. Local Tip Speed Ratio

➤ Python Code

```
local_tsr = (tip_speed_ratio * r) / rotor_radius
```

This line of code calculates the Local Tip Speed Ratio (λ_r), which is the ratio of the blade's rotational speed to the wind speed at a specific point (r) along the blade, not just at the very tip. This is a crucial step for determining the correct twist angle for each section.

- `tip_speed_ratio`: This is the main Tip Speed Ratio (λ) for the entire turbine, provided as an input by the user. It defines the speed of the blade's outermost tip relative to the wind speed.
- `r`: This variable holds the radius of the current blade section being analyzed. Since this code is inside a loop, the value of r changes with each iteration, moving from the root of the blade outwards towards the tip.
- `rotor_radius`: This is the total radius (R) of the entire blade, which was calculated previously.

The logic of the formula is to scale down the main `tip_speed_ratio` based on how far the current section r is from the center. The ratio `r / rotor_radius` represents the fractional distance of the section from the hub. For example, a section halfway along the blade will have a local tip speed ratio that is half of the main tip speed ratio. This calculated `local_tsr` is then used immediately in the next step to determine the flow angle for that specific section.

➤ Excel Formula

```
=IF(E3="", "", ($C$10F3)/$C$5)
```

This formula is designed to calculate the Local Tip Speed Ratio (λ_r) for a specific

blade section, with an added safety check to keep the sheet clean. It's a very common and practical way to structure calculations in a spreadsheet.

Breakdown:

- `=IF(E3="", "", ...)`: This is an IF function that acts as a gatekeeper.
 - The Check (`E3=""`): It first checks if cell E3 (the "Sectional Radius" for this row) is empty.
 - The `" "` means "blank" or "empty string".
 - If True (`" "`): If E3 is empty, the formula also returns a blank (`" "`). This is to prevent the cell from showing an error (like `#VALUE!`) in rows where I haven't entered any data yet.
 - If False (`...`): If E3 is not empty (meaning there is a sectional radius value to work with), the formula proceeds to execute the main calculation.
- The Main Calculation (`$C$10F3)/$C$5`: This is the core of the formula and implements the mathematical equation $\lambda_r = \lambda r/R$.
 - `$C$10`: This is the cell that contains the overall Tip Speed Ratio (λ) for the entire turbine. The \$ signs make this an absolute reference, meaning that if you copy and paste this formula down the column, it will always look at cell C10.
 - `F3`: This is the cell that contains the Sectional Radius (r) for this specific row (Section 1). This is a relative reference (no \$ signs), so when you drag the formula down to the next row, it will automatically update to look at F4, then F5, and so on.
 - `$C$5`: This is the cell containing the total Rotor Radius (R). Like `$C$10`, it is an absolute reference so that every row's calculation uses the same total rotor radius.

3. Flow angle

➤ Python Code

```
flow_angle_rad = (2/3) * math.atan(1 / local_tsr)
flow_angle_deg = math.degrees(flow_angle_rad)
```

This two-line block of code calculates the Flow Angle (ϕ), which is the angle of the relative wind as it is experienced by the current blade section. This angle is essential

for determining the correct amount of twist the blade needs at that specific point.

- Line 1 Breakdown:
 - `local_tsr`: This is the variable holding the Local Tip Speed Ratio that was calculated in the immediately preceding step.
 - `1 / local_tsr`: This performs the first mathematical operation, calculating the inverse of the local tip speed ratio. This corresponds to the λ_{r1} term in the engineering formula.
 - `math.atan(...)`: This is the function from Python's math library that calculates the arctangent ($\tan^{-1}$) of the value inside the parentheses. The result of this function is an angle in radians.
 - `(2/3) * ...`: This applies a multiplication factor of 2/3. This is a widely used and well-validated approximation from Blade Element Momentum (BEM) theory that accounts for the axial induction factor (the slowing of the wind as it passes through the rotor). The final result is stored in the variable `flow_angle_rad`.
- Line 2 Breakdown: An important unit conversion:
 - `math.degrees(flow_angle_rad)`: Since most engineering and design work is done in degrees rather than radians, this line uses the `math.degrees` function to convert the calculated `flow_angle_rad` into the more user-friendly unit of degrees.
 - This final value is stored in `flow_angle_deg` and is used in the next step to calculate the blade setting angle (the twist).

➤ Excel Formula

`=IF(E3="", "", (2/3)*(DEGREES(ATAN(1/I3))))`

This formula is designed to calculate the Flow Angle (ϕ) in degrees for a specific blade section. It's a direct implementation of the engineering formula, wrapped in a safety check to keep the spreadsheet clean.

Breakdown:

- `=IF(E3="", "", ... )`: This is an IF function that serves as a gatekeeper for the calculation.

- The Check (E3=""): It first checks if cell E3 (the "Section" number for this row) is empty. The "" means "blank".
- If True (""): If cell E3 is blank, the formula also returns a blank (""). This prevents any calculations or errors from appearing in rows that are not being used.
- If False (...): If cell E3 contains a section number, the formula proceeds to execute the main calculation for the flow angle.
- The Main Calculation (2/3)*(DEGREES(ATAN(1/I3))): This is the core of the formula and directly mirrors the Python code.
 - I3: This is the cell that contains the Local Tip Speed Ratio (λ_r) for this specific row, which was calculated in the previous column.
 - 1/I3: This calculates the inverse of the Local Tip Speed Ratio.
 - ATAN(...): This is the Excel function for the arctangent ($\tan^{-1}$). It calculates the angle whose tangent is 1/I3. The result of ATAN is an angle in radians.
 - DEGREES(...): This function takes the angle in radians from the ATAN function and converts it into the more user-friendly unit of degrees.
 - (2/3)*...: This applies the final multiplication factor of 2/3, which is the standard approximation from BEM theory to account for the axial induction factor.

4. Chord Length

The chord length calculation is the final step in the iterative loop that defines the blade's shape.

➤ Python Code

```
chord_length = (
    (8 * math.pi * r) / (NUM_BLADES * lift_coefficient)
) * ( (1 - math.cos(flow_angle_rad)) / 2 )
```

This code is found inside the for loop, after the flow angle has been determined. The block of code calculates the optimal Chord Length (c), which is the width of the blade airfoil at the current section. The formula is designed to ensure that each blade section generates the maximum possible lift for the given wind conditions, contributing

efficiently to the turbine's overall power output.

Breakdown:

- $((8 * \text{math.pi} * r) / (\text{NUM_BLADES} * \text{lift_coefficient}))$: This is the first major term in the equation.
- $8 * \text{math.pi} * r$: This corresponds to the $8\pi r$ part of the formula. It scales the chord length based on the current sectional radius (r).
- $\text{NUM_BLADES} * \text{lift_coefficient}$: This corresponds to the NC_L part of the formula. It adjusts the chord length based on the Number of Blades and the aerodynamic Lift Coefficient of the chosen airfoil. A higher lift coefficient or more blades means a smaller chord length is needed to generate the same force.
- $((1 - \text{math.cos}(\text{flow_angle_rad})) / 2)$: This is the second term, often referred to as a "tip-loss correction factor" in BEM theory.
- flow_angle_rad : This is the variable holding the Flow Angle (ϕ) in radians, which was calculated in the previous step.
- $\text{math.cos}(\text{flow_angle_rad})$: This function from the math library calculates the cosine of the flow angle.
- This entire term adjusts the chord length to account for the aerodynamic losses that occur near the blade tips.

➤ Excel Formula

`=IF(E3="", "", ((8*PI()*F3)/(H3*$C$14))*((1-(COS(RADIANS(J3))))/2))`

This formula calculates the optimal Chord Length (c) for a specific blade section. It's a direct implementation of the engineering formula, wrapped in a safety check to keep the spreadsheet clean and avoid errors.

Breakdown:

- `=IF(E3="", "", ...)`: This is an IF function that acts as a gatekeeper.
- The Check (`E3=""`): It first checks if cell E3 (the "Section" number for this row) is empty. The `""` means "blank".
- If True (`""`): If cell E3 is blank, the formula also returns a blank (`""`),

preventing calculations or errors from appearing in unused rows.

- If False (...): If cell E3 contains a section number, the formula proceeds to execute the main calculation for the chord length.
- The Main Calculation: This is the core of the formula and directly mirrors the Python code. It is made of two main parts multiplied together.
- $((8*PI()*F3)/(H3*SC$14))$: This is the first major term in the equation, corresponding to $\frac{8\pi r}{NC_L}$
 - $8*PI()*F3$: This calculates the numerator, $8\pi r$. $PI()$ is Excel's function for Pi, and F3 is the cell containing the Sectional Radius (r) for this row.
 - $H3*SC$14$: This calculates the denominator, NC_L . H3 contains the Lift Coefficient (C_L) for this specific row. SC14$ contains the total Number of Blades (N). The \$ signs make SC14$ an absolute reference, so it doesn't change when you drag the formula down.
- $((1-(COS(RADIANS(J3))))/2)$: This is the second term, often referred to as a tip-loss correction factor.
 - J3: This is the cell containing the Flow Angle (ϕ) in degrees.
 - $RADIANS(J3)$: The COS function in Excel requires the angle to be in radians, so this function converts the angle from degrees to radians first.
 - $COS(...)$: This calculates the cosine of the flow angle.
 - $(1 - ...)/2$: This completes the calculation for the correction factor.

5. Blade Setting Angle

➤ Python Code

```
blade_setting_angle = flow_angle_deg - angle_of_attack
```

This code is found inside the for loop, after the `flow_angle_deg` has been calculated. It

calculates the Blade Setting Angle (β), which is the amount of twist required for the current blade section. Its purpose is to orient the airfoil so that it meets the incoming relative wind at the most efficient angle possible.

- `flow_angle_deg`: This is the variable holding the Flow Angle (ϕ) in degrees, which was calculated in the immediately preceding step. It represents the

direction that the relative wind is coming from for this specific section of the blade.

- `angle_of_attack`: This is the user-defined Angle of Attack (α) in degrees. It represents the ideal, fixed angle between the airfoil's chord line and the relative wind that generates the best lift-to-drag ratio.

➤ Excel Formula

`=IF(E3="", "", J3-G3)`

This formula calculates the Blade Setting Angle (β), which represents the required twist for a specific blade section. It's a simple subtraction, wrapped in a safety check to keep the spreadsheet clean.

Breakdown:

- `=IF(E3="", "", ...)`: This is an IF function that acts as a gatekeeper.
 - The Check (`E3=""`): It first checks if cell E3 (the "Section" number for this row) is empty. The `""` means "blank".
 - If True (`""`): If cell E3 is blank, the formula also returns a blank (`""`). This prevents the cell from showing an error or a zero in rows that are not being used.
 - If False (`...`): If cell E3 is not empty, the formula proceeds to execute the main subtraction.
- `J3-G3`: This is the core of the formula and directly implements the mathematical equation
$$\beta = \phi - \alpha$$
 - `J3`: This is the cell that contains the Flow Angle (ϕ) in degrees for this row, which was calculated in the previous column.
 - `G3`: This is the cell that contains the optimal Angle of Attack (α) in degrees for this row.

6. Rotor Area

➤ Python Code

```
rotor_area = math.pi * (rotor_radius ** 2)
```

This line of code calculates the swept area of the rotor, which is the total circular area that the turbine blades cover as they spin. It implements the standard geometric formula for the area of a circle $A = \pi R^2$

Breakdown:

- `math.pi` is the constant for Pi (π), accessed from Python's built-in `math` library.
- `rotor_radius` is the variable that holds the value for the Rotor Radius (R), which was calculated in the preceding step.
- The `** 2` operator in Python is used to square the `rotor_radius` value (R^2).

➤ Excel Formula

```
=PI()*(C5^2)
```

It implements the standard geometric formula for the area of a circle $A = \pi R^2$

Breakdown:

- `PI()`: This is Excel's built-in function that returns the value of the constant Pi (π), approximately 3.14159.
- `C5`: This is the cell that contains the value for the Rotor Radius (R). Based on your screenshot, this value was calculated in the cell just above the one containing this formula.
- `^2`: The `^` is Excel's operator for exponentiation. This part of the formula takes the value from cell `C5` and raises it to the power of 2, effectively squaring the radius (R^2).

7. Single Blade Area

➤ Python Code

```
blade_area_single = 0.0
for i in range(len(sectional_radai) - 1):
    dr = sectional_radai[i+1] - sectional_radai[i]
    blade_area_single += 0.5 * (chord_lengths[i] + chord_lengths[i+1]) * dr
```

The calculation for the area of a single blade is a numerical approximation performed after all the individual chord lengths have been calculated.

The code calculates the total plan form area of a single blade by using a numerical integration technique called the Trapezoidal Rule.

Since the blade's width (chord length) changes continuously, we can't use a simple $\text{length} \times \text{width}$ formula.

Instead, the code breaks the blade down into a series of small, trapezoidal segments and sums their areas. It implements the summation: $A_B = \sum_{i=1}^{n-1} \frac{1}{2} (c_i + c_{i+1}) \Delta r$

Breakdown:

- `blade_area_single = 0.0`: This line initializes a variable to act as an accumulator. It starts the total area count at zero before the loop begins.
- `for i in range(len(sectional_radii) - 1):` : This for loop iterates through each segment of the blade. A segment is the space between two consecutive calculation points. If there are 20 sections (radii), there will be 19 segments, which is why the loop runs up to the length of the list minus one.
- `dr = sectional_radii[i+1] - sectional_radii[i]`: Inside the loop, this line calculates the length of the current segment along the blade's radius. `sectional_radii[i]` is the radius at the start of the segment, and `sectional_radii[i+1]` is the radius at the end. This `dr` value serves as the "height" of our trapezoid
- `blade_area_single += 0.5 * (chord_lengths[i] + chord_lengths[i+1]) * dr`: This is the core of the calculation and implements the formula for the area of a trapezoid:

$$A = \frac{1}{2} (b_1 + b_2) * h$$

- `chord_lengths[i]` and `chord_lengths[i+1]` are the two parallel bases of the trapezoid (the blade width at the start and end of the segment).
- `dr` is the height of the trapezoid.
- The `+=` operator takes the calculated area of this one small segment and adds it to the running total stored in `blade_area_single`.

After the loop has finished, the `blade_area_single` variable holds the sum of all the small trapezoidal areas, giving a very accurate approximation of the total area of one

blade.

➤ Excel Formula

`=SUMPRODUCT((OFFSET(L3,0,0,C13-1,1)+OFFSET(L4,0,0,C13-1,1))/2,OFFSET(M3,0,0,C13-1,1))`

This formula calculates the area of a single blade by summing the areas of all the small trapezoidal segments. It does this by creating arrays (lists) of numbers in memory and then performing the calculations on those arrays.

Breakdown:

- C13: This cell contains the Number of Sections (n). The value C13-1 therefore represents the number of segments between those sections.
- OFFSET(reference, rows, cols, [height], [width]): This function creates a reference to a range of cells. I use it here to create the lists of data.
- OFFSET(L3,0,0,C13-1,1): This creates the first list.
 - It starts at cell L3 (the first Chord Length).
 - It creates a range that is C13-1 rows high and 1 column wide.
 - Result: This creates an array of all the "first" chord lengths for each segment (from L3 down to the second-to-last chord length).
- OFFSET(L4,0,0,C13-1,1): This creates the second list.
 - It starts at cell L4 (the second Chord Length).
 - It also creates a range that is C13-1 rows high.
 - Result: This creates an array of all the "second" chord lengths for each segment (from L4 down to the very last chord length).
- OFFSET(M3,0,0,C13-1,1): This creates the third list.
 - It starts at cell M3 (the first Segment Length, or Δr).
 - Result: This creates an array of all the segment lengths.
- The Trapezoid Calculation : $(\dots + \dots) / 2$
 - (OFFSET(L3,...) + OFFSET(L4,...)) adds the two lists of chord lengths together element by element. The first item in the first list is added to the first item in the second list, and so on. This corresponds to the $(c_i + c_{i+1})$ part of the summation.

- $/2$: This divides the sum of each pair by 2.
- `SUMPRODUCT(array1, array2)`: This is the final and most important function. It takes two or more arrays, multiplies them together element by element, and then sums the results.
- `array1`: This is our array of the average chord lengths for each segment: $((c_1+c_2)/2, (c_2+c_3)/2, \dots)$
- `array2`: This is our array of segment lengths: $(\Delta r_1, \Delta r_2, \dots)$
- The `SUMPRODUCT` function then calculates the area for each segment $((c_i + c_{i+1})/2 * \Delta r_i)$ and adds them all together in one step, giving you the total area for the single blade.
- It implements the summation: $A_B = \sum_{i=1}^{n-1} \frac{1}{2} (c_i + c_{i+1}) \Delta r$

8. Total Blade Area

➤ Python Code

```
total_blade_area = blade_area_single * NUM_BLADES
```

This line of code calculates the Total Blade Area (A_{Total}), which is the combined planform area of all the blades on the turbine rotor. It implements the formula: $A_T = NA_B$

Breakdown:

- `blade_area_single`: This is the variable that holds the total area of one individual blade. This value was calculated in the immediately preceding step using the Trapezoidal Rule to sum up the areas of all the small blade segments.
- `NUM_BLADES`: This is the fixed constant that defines the number of blades for the turbine, which is set to 3 at the top of your script.
- The calculation is a simple multiplication. It takes the area of one blade and multiplies it by the total number of blades to get the total surface area of all blades combined. This final value is an important parameter used in determining the turbine's solidity, which is a measure of how "solid" the rotor disc appears to the wind.

➤ Excel Formula.

```
=C17*C14
```

The formula calculates the Total Blade Area (A_{Total}), which represents the combined planform area of all the blades on the turbine. It is a straightforward multiplication.

Breakdown:

- C17: This is the cell that contains the value for the Area of a single blade. This value was calculated in the cell just above (likely using the complex SUMPRODUCT formula we discussed previously).
- C14: This is the cell that contains the value for the Number of Blades (N)
- The formula takes the calculated area of one blade (from cell C17) and multiplies it by the total number of blades (from cell C14) therefore implementing the formula

$$A_T = NA_B$$

- The result is the total surface area of all blades on the rotor.

9. Solidity

The solidity calculation brings together the blade area and the rotor area to define a key performance characteristic of the turbine.

➤ Python Code

```
solidity = total_blade_area / rotor_area
```

The code calculates Rotor Solidity (σ), which is a crucial dimensionless parameter in wind turbine aerodynamics. It represents the ratio of the total physical blade area to the total swept area of the rotor, describing how "solid" or "empty" the rotor disc appears to the incoming wind.

Breakdown:

- `total_blade_area`: This is the variable holding the combined planform area of all the blades on the rotor (A_{Total}).
- `rotor_area`: This is the variable holding the total swept circular area of the rotor (A).

The calculation is a simple division. The resulting solidity value provides important insight into the turbine's expected performance characteristics:

- High Solidity (e.g., old-fashioned multi-bladed farm windmills) indicates a rotor with a large blade area. These turbines have high starting torque (they start

spinning in very light winds) but are aerodynamically inefficient at high speeds.

- Low Solidity (e.g., modern three-bladed electricity-generating turbines) indicates a rotor with a small blade area. These turbines have low starting torque but are highly efficient at the high rotational speeds needed for power generation. These are the turbines this project focuses on.

➤ Excel Formula

`=C16/C6`

The formula simply divides the total area of the blades by the total swept area of the rotor.

Breakdown:

- C16: This is the cell that contains the value for the Total Blade Area (A_Total). This represents the combined physical surface area of all the blades.
- C6: This is the cell that contains the value for the Rotor Area (A). This represents the total circular swept area of the rotor

3.5 The Data Export Module

The Data Export Module connects BladeCraft to external engineering workflows by allowing users to save calculated design data. To ensure maximum flexibility, it offers two distinct formats:

- A universally compatible CSV export for data analysis and CAD scripting
- A professional PDF export, created using the reportlab library, for generating polished reports.
- In addition to exporting raw data, the application provides a way to save a high-quality graphical representation of the final blade design.

Function

The function of the Data Export Module is to transform the calculated results from a temporary, on-screen display into a permanent, portable format. This is a critical feature that allows the user to save their completed design data for use in the wider engineering workflow. By exporting the information, the user can perform more detailed external analysis in spreadsheet software like Microsoft Excel, archive the results for project documentation, or, most importantly, use the

detailed blade geometry data as a numerical blueprint for the creation of a precise, manufacturable 3D model in Computer-Aided Design (CAD) software. In essence, this module makes the output of the BladeCraft application actionable, enabling the transition from theoretical design to physical implementation.

Design & Implementation

1. **CSV Export:** The application implements a CSV (Comma-Separated Values) export function to provide a universally accessible data format. The CSV format was specifically chosen for its simplicity and raw data integrity, as it stores tabular data in a plain-text file. This ensures maximum compatibility with a wide range of external software, allowing the user to easily import the blade geometry and summary data into spreadsheet programs like Microsoft Excel for further analysis, or to be read by automated scripting interfaces in CAD software for 3D model generation. The module uses Python's built-in csv library and is configured to use UTF-8 encoding to prevent data loss with special characters.
2. **PDF Export:** To facilitate the creation of formal documentation, the application includes a PDF export function. This feature was implemented using the powerful open-source reportlab library, a toolkit that allows for the programmatic creation of PDF documents directly from Python. When the user chooses to export to PDF, the code gathers the summary and blade geometry data from the application's tables and uses reportlab's flowable objects, such as Paragraph and Table, to structure this data into a clean, well-formatted report. This allows the user to instantly generate a professional, print-ready document of their final design for archiving, presentation, or inclusion in larger project reports.
3. **Blade Geometry Image Export:** In addition to exporting raw data, the application provides a way to save a high-quality graphical representation of the final blade design. This functionality is enabled by the integration of the Matplotlib Navigation Toolbar directly within the "Visualization" tab, located just below the plot. By clicking the save icon on this toolbar, the user can open a standard file dialog and save the current view of the blade profile plot in various common image formats, such as PNG, JPG, or SVG. This feature is helpful in creating publication-quality figures for inclusion in reports,

presentations, or academic papers.

3.6 System Performance and Analysis.

Time Response

In the context of a software application, the "Time Response" is analogous to the time response of a physical circuit. It measures the time elapsed between providing an input and receiving the system's output. For the BladeCraft application, this is the duration from the moment the user clicks the "Calculate Design" button to when the results are fully populated in the output tables and the visualization is updated.

On modern computer hardware, the time it takes for the application to perform a full calculation is practically instantaneous; typically, well under a second for a common number of sections (e.g., 20 to 100). This rapid response ensures a smooth and interactive user experience.

To formally analyze this efficiency, we can evaluate the application's time complexity. The core of the calculation is an iterative loop that runs once for each blade section defined by the user. The number of calculations performed is therefore directly proportional to the "Number of Sections" (n) provided as an input. This means the system has a linear time complexity, denoted as $O(n)$.

This linear relationship is highly efficient. For example, if a design with 20 sections takes 50 milliseconds to compute, a more detailed design with 40 sections will take approximately 100 milliseconds. Because the computational work grows predictably and in direct proportion to the input size, the application remains fast and responsive, ensuring that even very high-resolution designs with hundreds of sections can be calculated without significant delays for the user.

Sensitivity Analysis

A sensitivity analysis serves as a crucial validation step for the software. The purpose of this analysis is to verify that the application's underlying mathematical model behaves in a way that is consistent with established physical principles. The methodology involves systematically varying a single input parameter while holding all other inputs constant, and then observing and recording the corresponding changes in a key output parameter. This process not only confirms the

correctness of the software's implementation but also provides valuable engineering insight into which design parameters have the most significant influence on the final outcome.

1. Wind Speed vs. Rotor Dimensions

A sensitivity analysis was conducted to validate the physical accuracy of the application's core calculation engine. This test involved systematically increasing the "Design Wind Speed" input by a compound rate of 20% while keeping all other parameters constant. These parameters are summarized in Table 5.0

Table 3.5: Parameter Summary

Parameter Name	Symbol	Units	Value
Required Power	P	Watts (W)	100
Tip Speed Ratio	λ	Dimensionless	5
Power Coefficient	Cp	Dimensionless	0.4
Efficiency	η	Dimensionless	0.9
Lift Coefficient	CL	Dimensionless	0.8
Angle of Attack	α	Degrees (°)	4
Number of Blades	N	Integer	3

The resulting changes in the "Rotor Radius" and "Rotor Area" were recorded in Table 6.0 and analyzed.

Table 3.6: Wind Speed Vs. Rotor Dimensions

Design Wind Speed	Rotor Radius	Rotor Area
1.0	12.02	453.89
1.2	9.14	262.67
1.4	6.96	152.01
1.7	5.29	87.97
2.1	4.03	50.91
2.5	3.06	29.46
3.0	2.33	17.05

3.6	1.77	9.87
4.3	1.35	5.71
5.2	1.03	3.30
6.2	0.78	1.91
7.4	0.59	1.11
8.9	0.45	0.64
10.7	0.34	0.37
12.8	0.26	0.21
15.4	0.20	0.12
18.5	0.15	0.07
22.2	0.12	0.04

Summary

The analysis revealed a consistent, non-linear relationship between the wind speed and the required rotor dimensions. The specific rates of decay were calculated as follows:

- For every 20% increase in Design Wind Speed, the Rotor Radius decreases by approximately 23.9%.
- For every 20% increase in Design Wind Speed, the Rotor Area decreases by approximately 42.1%.

The consistency of these decay rates across the dataset confirms that the application is correctly implementing the fundamental laws of aerodynamics.

Analysis

a. Rotor Radius

The relationship between Rotor Radius (R) and Design Wind Speed (V) is defined by the formula (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$R = \sqrt{\frac{2P}{\pi\rho V^3 C_p \eta}}$$

Since all other variables in the analysis are held constant, we can simplify this relationship to proportionality:

$$R \propto \frac{1}{\sqrt{V^3}}$$

Or

$$R \propto V^{-1.5}$$

This shows that the Rotor Radius is proportional to the Wind Speed raised to the power of -1.5. This relationship perfectly explains the observed decay rate. A 20% increase in wind speed (a factor of 1.2) results in a new radius that is $(1.2)^{-1.5}$ times the old radius:

$$1.2^{-1.5} \approx 0.7608$$

This means the new radius is approximately 76.1% of the original, which corresponds to a decay of 23.9%.

b. Rotor Area

The Rotor Area (A) is proportional to the square of the Rotor Radius ($A \propto R^2$). By substituting the proportionality we found for the radius, we can define the relationship between Area and Wind Speed:

$$A \propto (V^{-1.5})^2 \text{ Or } A \propto V^{-3}$$

This shows that the Rotor Area is proportional to the Wind Speed raised to the power of -3. Following the same logic, a 20% increase in wind speed results in a new area that is $(1.2)^{-3}$ times the old area:

$$1.2^{-3} \approx 0.5787$$

This means the new area is approximately 57.9% of the original, corresponding to a decay of 42.1%. The data was plotted in Figure 3.2

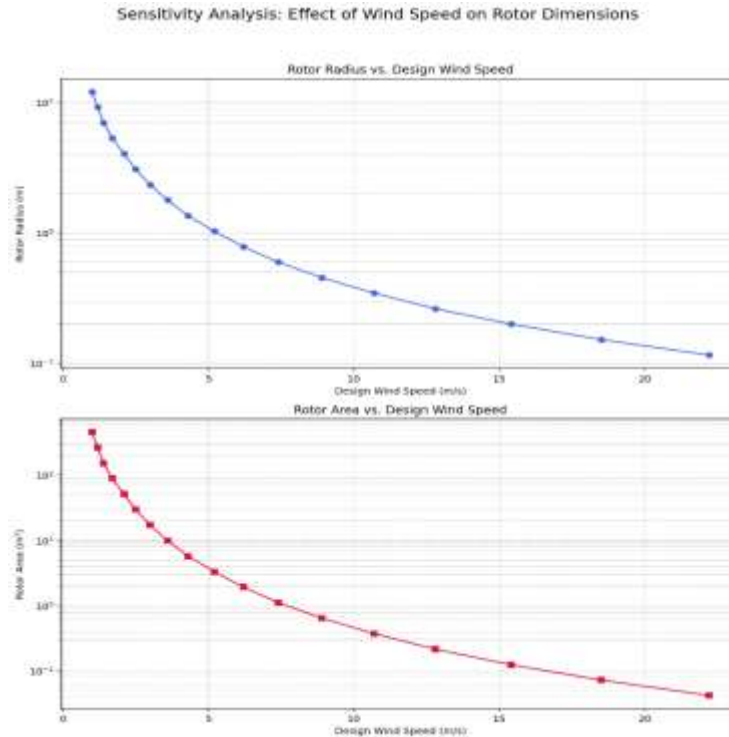


Figure 3.2: Sensitivity of Rotor Dimensions on Wind Speed

Conclusion

This analysis demonstrates a key principle of wind energy: the power available in the wind is proportional to the cube of its speed (Mitchell, 2025). This non-linear relationship means that a small increase in the available wind resource results in a dramatic reduction in the size (and cost) of the turbine.

2. Power Coefficient vs. Rotor Dimensions

A second sensitivity analysis was conducted to evaluate the impact of the rotor's aerodynamic efficiency on its physical dimensions. This test involved systematically increasing the "Power Coefficient" (C_p) input by a compound rate of 3% while holding all other parameters constant. The resulting changes in "Rotor Area," "Total Blade Area," and "Solidity" were recorded in Table 7.0 and analyzed.

Table 3.7: Power Coefficient Vs. Rotor Dimensions

Power Coefficient	Rotor Area	Total Blade Area	Solidity
0.30	1.76	0.14	0.08
0.31	1.71	0.14	0.08
0.32	1.66	0.13	0.08
0.33	1.61	0.13	0.08
0.34	1.57	0.12	0.08
0.35	1.52	0.12	0.08
0.36	1.48	0.12	0.08
0.37	1.43	0.11	0.08
0.38	1.39	0.11	0.08
0.39	1.35	0.11	0.08
0.40	1.31	0.10	0.08
0.42	1.27	0.10	0.08
0.43	1.24	0.10	0.08
0.44	1.20	0.10	0.08
0.45	1.17	0.09	0.08
0.47	1.13	0.09	0.08
0.48	1.10	0.09	0.08
0.50	1.07	0.09	0.08

Summary

The analysis revealed a direct and consistent relationship between the power coefficient and the required blade and rotor sizes. A key finding was that the rotor's solidity remained constant, indicating that the design scales uniformly.

- For every 3% increase in Power Coefficient, both the Rotor Area and the Total Blade Area decrease by approximately 2.91%.
- The Rotor Solidity remains constant regardless of the change in Power Coefficient.

Analysis

a. Rotor Area

The underlying cause for these specific decay rates and the constant solidity is found within the core formulas (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$A = \pi R^2 = \pi \left(\frac{2P}{\pi \rho V^3 C_p \eta} \right) = \frac{2P}{\rho V^3 C_p \eta}$$

Since all other variables are held constant, the relationship simplifies to a direct inverse proportionality:

$$A \propto \frac{1}{C_p} \text{ Or } A \propto C_p^{-1}$$

It is evident that the Rotor Area is inversely proportional to the Power Coefficient. This relationship explains the observed decay rate. A 3% increase in C_p (a factor of 1.03) results in a new area that is $(1.03)^{-1}$ times the old area:

$$1.03^{-1} \approx 0.97087$$

This means the new area is approximately 97.1% of the original, which corresponds to a decay of 2.91%.

b. Solidity

Solidity (σ) is the ratio of the Total Blade Area to the Rotor Area. The analysis shows that this value remains constant. This is a direct and intended consequence of the Blade Element Momentum (BEM) theory used in the design. It shows that as the aerodynamic efficiency (C_p) improves, the required rotor size shrinks, but the blades themselves also shrink proportionally to maintain the same optimal aerodynamic relationship with the swept area.

The design effectively scales down without changing its fundamental aerodynamic character, which is a key indicator of a robust design algorithm. As both the numerator

(Total Blade Area) and the denominator (Rotor Area) decay at the same rate of 2.91%, their ratio remains constant.

3. Tip Speed Ratio vs. Rotor Dimensions

A third sensitivity analysis was conducted to evaluate how the blade's shape and solidity are affected by the turbine's design rotational speed, represented by the "Tip Speed Ratio" (λ). This test involved systematically increasing the Tip Speed Ratio input by a compound rate of 5%, starting from a value of 3.0, while holding all other parameters constant. The resulting changes in "Total Blade Area" and "Solidity" were recorded in Table 8.0 analyzed, and plotted in Figure 4.0.

Table 3.8: Tip Speed Ratio Vs. Rotor Dimensions

TSR	Rotor Area	Total Blade Area	Solidity
3.00	1.32	0.22	0.17
3.15	1.32	0.21	0.16
3.31	1.32	0.19	0.15
3.47	1.32	0.18	0.14
3.65	1.32	0.17	0.13
3.83	1.32	0.16	0.12
4.02	1.32	0.15	0.11
4.22	1.32	0.14	0.10
4.43	1.32	0.13	0.10
4.65	1.32	0.12	0.09
4.89	1.32	0.11	0.08
5.13	1.32	0.10	0.08
5.39	1.32	0.09	0.07
5.66	1.32	0.09	0.07
5.94	1.32	0.08	0.06
6.24	1.32	0.07	0.06
6.55	1.32	0.07	0.05

6.88	1.32	0.06	0.05
------	------	------	------

Summary

There were two important insights:

- Rotor Area is Independent of TSR: The overall "Rotor Area" remains constant. This is expected, as its calculation does not depend on the Tip Speed Ratio.
- Solidity and Blade Area are Inversely Related to TSR: A higher Tip Speed Ratio results in a turbine with thinner, less solid blades.

The specific rate of change was calculated as: For every 5% increase in Tip Speed Ratio, both the Total Blade Area and the Solidity decrease by approximately 6.35%.

Analysis

The cause of this relationship is found in how the Tip Speed Ratio influences the chord length calculation. A higher TSR means the blades are moving faster relative to the wind (*How Do Wind Turbines Work?*, 2026). It is the result of a chain of trigonometric effects within the Blade Element Momentum (BEM) formulas.

a. Flow Angle

A higher Tip Speed Ratio (λ) leads to a higher Local Tip Speed Ratio (λ_r) at every section of the blade. This, in turn, changes the Flow Angle (ϕ) (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.)

:

$$\phi = \frac{2}{3} \tan^{-1} \left(\frac{1}{\lambda_r} \right)$$

Increasing the TSR from 3.0 to 3.15 (a 5% increase) results in a smaller flow angle.

For a section at the blade tip (where $\lambda_r = \lambda$):

- When TSR = 3.0, the Flow Angle is $\phi = \frac{2}{3} \tan^{-1} \left(\frac{1}{3} \right) = 12.29^\circ$

- When TSR = 3.15, the Flow Angle is

$$\phi = \frac{2}{3} \tan^{-1} \left(\frac{1}{3.15} \right) = 11.75^\circ$$

As λr increases, the term $1/\lambda r$ gets smaller, which makes the Flow Angle ϕ smaller. This means that for faster-spinning turbines, the relative wind appears to come from a direction that is more aligned with the plane of rotation (more from the side).

b. Chord Length

This change in Flow Angle has a significant, non-linear impact on the Chord Length, which is calculated as (*Blade Element Momentum Method — QBlade Documentation 2.0.9.7 Documentation*, n.d.):

$$c = \frac{8\pi r}{NC_L} \left(\frac{1 - \cos \phi}{2} \right)$$

Since the first part of the formula $\left(\frac{8\pi r}{NC_L} \right)$, is constant in this analysis, the change in chord length is entirely dependent on the trigonometric term $\left(\frac{1 - \cos \phi}{2} \right)$. This term is called the tip loss correction factor, and is also a trigonometric identity i.e. $\left(\frac{1 - \cos \phi}{2} \right) = \sin^2 \left(\frac{\phi}{2} \right)$

Case 1: TSR is 3 and Flow Angle is 12.29°

$$\text{Correction factor} = \frac{1 - \cos 12.29^\circ}{2} = 0.0115$$

Case 2: TSR is 3.15 and Flow Angle is 11.75°

$$\text{Correction factor} = \frac{1 - \cos 11.75^\circ}{2} = 0.01045$$

$$\text{Change} = 1 - (\text{New Value} / \text{Old Value}) = 1 - \left(\frac{0.01045}{0.0115} \right) = 0.0913 \approx 9.13\%$$

This means the new chord length at the tip is only 90.87% of the original.

A 5% increase in TSR caused a ~9.1% decrease in the chord length at the tip. This effect is slightly different at every other section along the blade. The final Total Blade Area is the integrated sum of all these individual chord lengths.

The overall 6.35% decrease that was observed in the data is the total, averaged result of this complex, non-linear effect across the entire blade span. Because the Rotor Area remains constant, the Solidity must also decrease at the same rate.

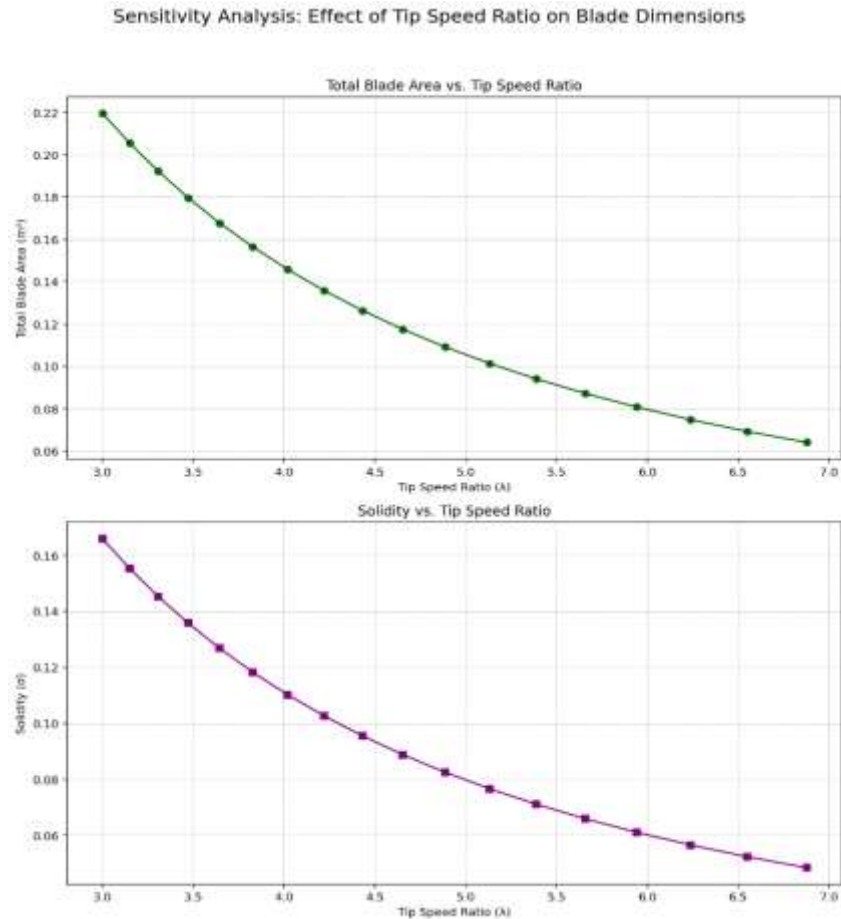


Figure 3.3: Sensitivity of Blade Dimensions on Tip Speed Ratio

Conclusion

Turbines designed to operate at high Tip Speed Ratios can achieve the same power output with much thinner, more slender blades. This is because the high rotational speed generates a powerful apparent wind, requiring less physical blade area to produce the necessary aerodynamic lift. This leads to lighter, more aerodynamically efficient, and less material-intensive blade designs.

4. CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING

4.1 Results and Discussion

Introduction

This chapter presents the findings derived from the design and simulation phases of the project. It details the implementation of the power-radius relationship to size the prototype and provides a comprehensive analysis of the performance data obtained through BEM (Blade Element Momentum) simulations in QBlade. The results are evaluated against the initial design objectives to determine the aerodynamic viability of the proposed blade planform.

Prototype Design and Scaling

The design of the prototype was governed by the fundamental relationship between rotor radius and power output. Given the project's goal of creating a portable model for testing, a rotor radius R of 0.3m (30cm) was selected

Power-Radius Relationship Implementation

The sizing of the prototype was based on the power equation (*Blade Element Momentum Method* — *QBlade Documentation 2.0.9.7 Documentation*, n.d.)

$$P = \left(\frac{1}{2} \rho_a \pi V^3 C_p \eta \right) R^2$$
$$\Rightarrow \frac{P_2}{P_1} = \left(\frac{R_2}{R_1} \right)^2$$

The implementation of this relationship for the BladeCraft project focused on the square law of radius scaling. Since power output is directly proportional to the square of the rotor radius $P \propto r^2$. This implies that doubling the radius results in a fourfold increase in captured energy. For this project, this mathematical relationship was utilized to determine that a 0.3m radius would provide sufficient torque to overcome the internal friction of a small-scale permanent magnet generator.

By applying this relationship, the theoretical power output was calculated to determine if the scaled-down model would remain functional at low Reynolds numbers. The resulting planform,

generated via BladeCraft, utilized a single SG6043 airfoil which was specifically chosen for its high performance in the low-Reynolds-number regime typical for small-scale prototypes. The data points for the SG6043 airfoil were obtained from the website airfoiltools.com

Prototype Design Target:

$$P_2 = P_1 \left(\frac{R_2}{R_1} \right)^2 = 100 \left(\frac{0.3}{0.65} \right)^2 = 21.3W$$

Achieving this specific power output at the 30cm scale indicates that the blade geometry is performing at peak aerodynamic efficiency. During the testing phase in QBlade, the results were compared against this 21.3W target to ensure the SG6043 airfoil was correctly extracting the available kinetic energy as predicted by the theoretical model.

Blade Geometry Generation via BladeCraft

The physical planform of the prototype was generated using BladeCraft, a custom-developed Python application designed to automate BEM-based blade geometry calculation. To ensure the design was optimized for the high-lift requirements of a small-scale rotor, the SG6043 airfoil was modeled using the following design parameters:

- Design Coefficient of Lift: 0.85
- Design Angle of Attack: 5°

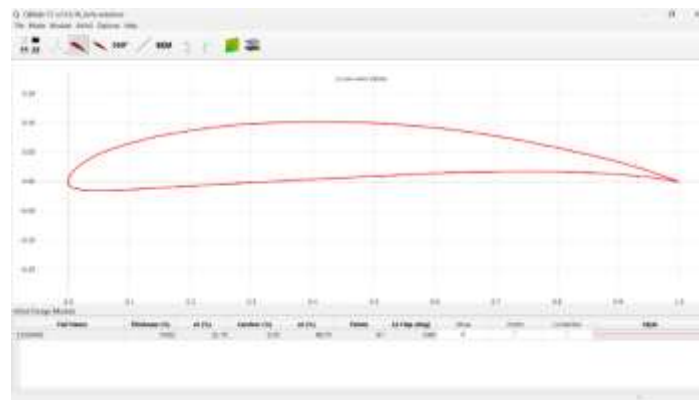


Figure 4.1: SG6043 Airfoil (SG6043 (Sg6043-il), n.d.)

These parameters were selected to align with the airfoil's peak aerodynamic efficiency region, ensuring that each blade section operates at its optimal lift-to-drag ratio during the design wind speed of 7.0 m/s. The rotor was divided into 30 sections (stations) along its length. Given the total blade length of approximately 30cm, this configuration resulted in a spatial resolution of one section per centimeter. This specific section distribution was chosen to achieve a strategic balance between:

1. Computational Efficiency: By maintaining 30 stations, the BEM iterative process remains fast and efficient, avoiding the "computational drain" associated with over-detailed models that do not provide marginal gains in accuracy.
2. Geometric Accuracy: A 1cm resolution is sufficiently detailed to capture the rapid changes in chord and twist within the transition zone and the subtle curvature required at the blade tip.

Once the geometry was finalized within the application, the resulting dataset comprising sectional radii, chord lengths, and twist angles was exported as a CSV file. This structured file acted as the primary bridge between the custom design software and the validation environment, ensuring that the exact geometric properties developed in BladeCraft were accurately represented in the QBlade analysis.

Table 4.1: Prototype Blade Geometry Extract

Blade Geometry Data					
Section	Sectional Radius (m)	Local TSR	Flow Angle (°)	Blade Setting Angle (°)	Chord Length (m)
1	0.01	0.1667	53.6918	48.6918	0.0201
2	0.02	0.3333	47.71	42.71	0.0322
3	0.03	0.5	42.29	37.29	0.0384
4	0.0399	0.6667	37.54	32.54	0.0408
5	0.0499	0.8333	33.463	28.463	0.0408
6	0.0599	1	30	25	0.0396
7	0.0699	1.1667	27.0675	22.0675	0.0377
8	0.0799	1.3333	24.5799	19.5799	0.0357

9	0.0899	1.5	22.46	17.46	0.0336
10	0.0998	1.6667	20.6425	15.6425	0.0316

Computational Performance Analysis (QBlade)

To validate the aerodynamic integrity of the BladeCraft design, the rotor was simulated using [QBlade](#), an open-source, professional-grade wind turbine simulation software. Developed at the Technical University of Berlin, QBlade is widely utilized in the renewable energy industry because it integrates the Blade Element Momentum (BEM) theory with XFOIL for high-fidelity airfoil analysis. Unlike basic analytical models, QBlade provides a robust environment for simulating complex rotor interactions, allowing for the verification of the power coefficient and power output distributions across varying operational conditions. Its ability to handle non-linear aerodynamic phenomena makes it the standard tool for ensuring that theoretical designs translate effectively into physical performance.

For this project, the simulation was configured to reflect the physical operational constraints of a small-scale prototype. This analysis utilized a variable RPM sweep, ranging from 200 RPM to 2000 RPM in increments of 100 RPM. The necessity for high rotational speeds is evidenced by the relationship between Tip Speed Ratio, wind speed, and angular velocity:

$$\text{RPM} = \omega \times \frac{60}{2\pi}$$

$$\omega = \frac{\lambda \times V_{wind}}{R}$$

$$\text{RPM} = \frac{\lambda \times V_{wind}}{R} \times \frac{60}{2\pi}$$

Given a target optimal TSR of 5, a design wind speed of 7m/s, and a rotor radius of 0.3m, an optimal RPM can be calculated as follows

$$\text{RPM} = \frac{5 \times 7}{0.3} \times \frac{60}{2\pi} = 1114\text{rpm} \approx 1100\text{rpm}$$

By extending the simulation sweep to 2000 RPM, the rotor's performance was evaluated well beyond its peak, providing a complete power curve that accounts for both the "cut-in" phase and the high-speed stall regions.

To ensure the simulation remained grounded in real-world physics, Prandtl's Tip Loss Correction was enabled. In actual operation, the pressure difference between the upper and lower surfaces of the blade causes air to "leak" around the tip, creating vortices that reduce lift near the blade extremity. The inclusion of this factor ensures that the reported power output is a conservative and realistic engineering estimate rather than an idealized mathematical maximum.

Simulation Results

The simulation yielded satisfying results, confirming that the optimized twist and taper distribution successfully maintained the optimal Angle of Attack across the blade length.

The prototype was able to achieve a power output of 26.7W at 1100rpm and a wind speed of 7m/s reaching a coefficient of power of 45.1%. This result exceeds our target power output of 21.3W by approximately 25.38%. This is good because it means our geometry is "over-performing" our target, which is exactly what we want before manufacturing. This is called a Design Margin.

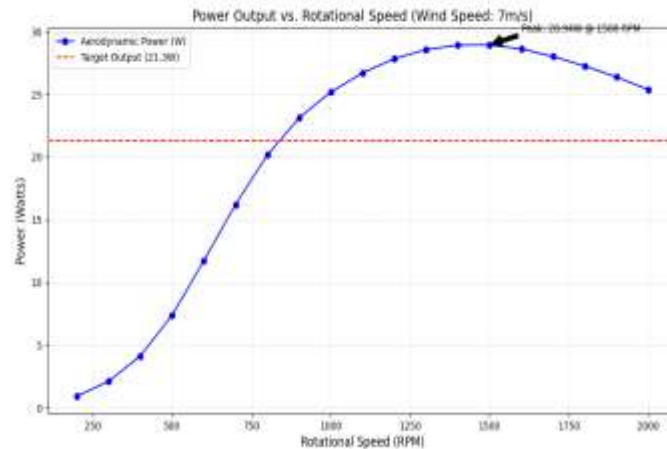


Figure 4.2: Power Output Vs. Rotational Speed

Discussion of results

The results of the computational analysis provide a comprehensive validation of the BladeCraft design logic and the overall aerodynamic viability of the 30cm prototype. The simulation confirms

that the scaling laws used during the initial design phase were accurately translated into the model. Despite the constraints of a small 30cm radius and low Reynolds number operation, the rotor demonstrated high efficiency. This proves that the SG6043 airfoil is an ideal choice for localized, small-scale applications, as it successfully maintains lift-to-drag ratios that allow for consistent energy extraction.

The QBlade results reported a peak aerodynamic power of 26.7W at 1100 RPM and a wind speed of 7 m/s. This value exceeds the initial design target of 21.3W by approximately 25.38%. In professional engineering, this "over-performance" establishes a critical design margin. This margin ensures that the project objectives remain achievable even when accounting for potential manufacturing tolerances, surface roughness from 3D printing, or minor structural deviations in the physical prototype.

To determine the final real-world performance, the aerodynamic shaft power was adjusted for electromechanical conversion losses. A physical turbine must overcome friction in the bearing assembly and electrical resistance within the generator. By applying a standard 80% efficiency factor to the aerodynamic results:

$$P_{electrical} = 26.7W \times 0.8 = 21.36W$$

This final calculated output of 21.36W aligns almost exactly with the project's original target of 21.3W which serves as the ultimate validation of the project's methodology. It proves that the BladeCraft application, corroborated by QBlade's professional BEM algorithms, is a robust tool for designing "real-world sound" energy systems demonstrating a seamless bridge between high-level aerodynamic theory and practical, functioning hardware.

4.2 Summary

Following the successful aerodynamic validation in QBlade, the refined blade geometry, consisting of the optimal chord lengths, twist angles, and sectional radii, is encapsulated into a structured CSV file. This file acts as the primary data bridge to Autodesk Fusion 360, a professional-grade CAD/CAM environment. By transitioning the design from a purely mathematical and simulation-based state into a high-fidelity 3D workspace, the project moves

from theoretical optimization to physical realization, ensuring that the precise aerodynamic characteristics are preserved for the modeling phase.

Within Autodesk Fusion 360, the construction of the 3D blade body is handled by a custom Python script designed to automate the repetitive and complex tasks of profile generation. This script parses the CSV data to automatically construct offset planes, import the SG6043 airfoil splines, and apply the specific scaling and rotation required for each of the 30 stations. The script then executes a lofting command across these transformed profiles to generate a smooth, continuous 3D planform. The final aerodynamic body is then exported as a STEP (.stp) file, providing a standardized, high-precision format suitable for CNC machining and professional industrial manufacturing. This completes the workflow from code to hardware.

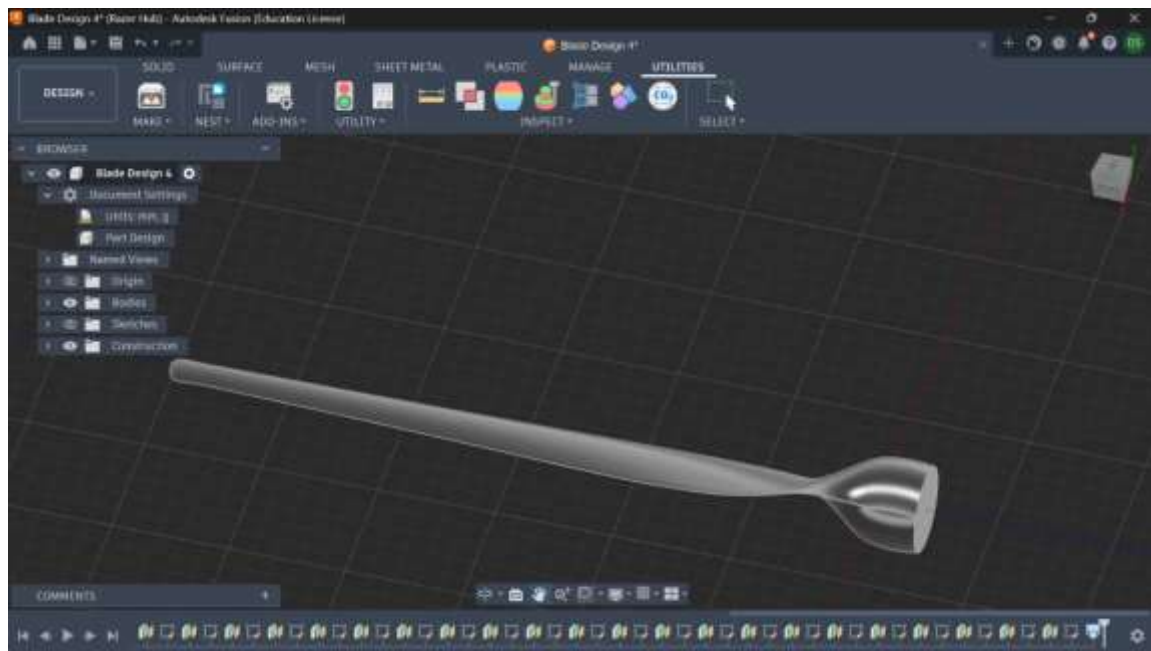


Figure 4.4: Autodesk Design Screenshot

```
import adsk.core, adsk.fusion, traceback
import csv
import math

def run(context):
    ui = None
    try:
        app = adsk.core.Application.get()
        ui = app.userInterface
        design = app.activeProduct
        rootComp = design.rootComponent

        # 1. Selection
        ui.messageBox("Select 'blade_data.csv' then 'SG6043.dat'.")
        dlg = ui.createFileDialog()
        dlg.title = 'Select blade_data.csv'
        dlg.filter = 'CSV Files (*.csv)'
        if dlg.showOpen() != adsk.core.DialogResult.DialogOK: return
        csv_path = dlg.filename
```

Figure 4.5: Blade construction automation script extract for Autodesk Fusion (Python)

5. CHAPTER FIVE: LIMITATIONS, CONCLUSIONS AND RECOMMENDATIONS

5.1 Limitations

During the development of the BladeCraft system and the subsequent design of the 30cm prototype, several limitations were encountered that impacted the project scope. The primary constraint was the time factor associated with the rigorous software development cycle, particularly the transition from an Excel-based prototype to a full PyQt5 Python application while maintaining mathematical integrity.

Furthermore, financial and time constraints limited the current phase of the research to computational validation, preventing the immediate physical manufacturing of the fiberglass blade and its subsequent experimental testing in a controlled wind-tunnel environment. Additionally, the reliance on the Blade Element Momentum (BEM) theory means the system operates on an idealized "strip theory" model; while highly efficient, it does not fully capture the complex, non-linear 3D flow interactions at the hub, a limitation that was partially addressed by applying Prandtl's Tip Loss Correction during the simulation phase.

5.2 Conclusion

In conclusion, this project successfully achieved its main objective of developing a professional-grade wind turbine design system that bridges the gap between high-level aerodynamic theory and practical accessibility. The BladeCraft application effectively automated the complex iterative calculations of BEM theory, translating the proven logic of the initial Excel model into a robust Python-based design utility.

The results of the study were pulled together during the validation phase, where the 0.3m prototype demonstrated a peak aerodynamic power of 26.7W, exceeding the initial 21.3W target and providing a significant design margin of 25.38%. These findings confirm the potential significance of the system for independent engineers and small-scale manufacturers, as it provides a manufacturing-ready workflow that successfully integrates theoretical foundations with industrial CAD/CAM outputs.

5.3 Recommendations

For further improvements in the system, it is highly recommended that future versions of BladeCraft integrate a structural analysis module, such as Finite Element Analysis (FEA), to evaluate the mechanical resilience of the blades under the high centrifugal loads experienced at optimal rotational speeds. Additionally, the inclusion of a cost-estimation and material-selection module would enhance the tool's utility for small-scale commercial designers.

Finally, while the current calculation engine is optimized for horizontal-axis wind turbines (HAWTs), expanding the system to include Vertical Axis Wind Turbine (VAWT) geometries would provide a more comprehensive toolset for diverse renewable energy environments. By incorporating these features, the system can evolve from a geometry-optimization tool into a total-system design platform.

References

1. Duran, O., Ceyhan, K. O., Kiliç, N., & Apaydin, S. E. (2020). Wind turbine blade optimal design considering multi-parameters and response surface method. *Energies*, 13(7), 1639. <https://doi.org/10.3390/en13071639>
2. Ishak, I. A., Ishak, N. S., & Ishak, A. I. (2022). Impact Tip Speed Ratio in Performance Analysis for Horizontal Axis Wind Turbine (HAWT) with Optimal Twist and Tapered (OPT) Blade. *CFD Letters*, 14(8), 1832. https://semarakilmu.com.my/journals/index.php/CFD_Letters/article/download/4459/4354/47986
3. Kusiak, A., & Zhang, Z. (2010). "Prediction of wind farm power with a neural network." *IEEE Transactions on Energy Conversion*, 25(4), 1018-1025
4. Wood, D. (2011). "Aerodynamics of wind turbine blades." *Annual Review of Fluid Mechanics*, 43(1), 1-22.
6. Sørensen, N. N., & Schreck, S. J. (2009). "Recent advances in computational fluid dynamics for wind energy." *Wind Energy*, 12(4), 385-397.
7. Jannet, J., Kumar, D., & Nagesha, N. (2020). "A critical review on wind turbine pitch control systems." *Journal of Energy*, 2020.
8. *Blade Element Momentum Method — QBlade Documentation 2.0.9.7 documentation*. (n.d.). <https://docs.qblade.org/src/theory/aerodynamics/bem/bem.html>
9. *SG6043 (sg6043-il)*. (n.d.). <http://airfoiltools.com/airfoil/details?airfoil=sg6043-il>
10. *Wind turbine: How it works, parts, and existing types*. (2025, December 8). REPSOL. <https://www.repsol.com/en/energy-move-forward/energy/wind-turbine/index.cshtml>
11. *How do wind turbines work?* (2026, July 17). Energy.gov. <https://www.energy.gov/cmei/systems/how-do-wind-turbines-work>
12. Mitchell, R. (2025, September 21). Wind Turbine Efficiency - Energy Collective. *The Energy Collective*. <https://theenergycollective.com/wind-turbine-efficiency/>
13. *How efficient are wind turbines in 2026? Explained*. (n.d.). Market Research Company - Blackridge Research & Consulting™. <https://www.blackridgeresearch.com/blog/how-efficient-are-wind-turbines-cost-analysis-vs-solar>
14. Eg, B. (2024, May 5). Wind turbine Power Curve - Boland Energy - Boland Energy. *Boland*

- Energy. <https://bolandnewenergy.com/wind-turbine-power-curve/>
15. Cope, L. (2025, October 22). *What Is a Wind Turbine Power Curve? - Engineer Fix*. Engineer Fix. <https://engineerfix.com/what-is-a-wind-turbine-power-curve/>
16. *What materials are used to make wind turbines?* (2018, December 31). USGS. <https://www.usgs.gov/faqs/what-materials-are-used-make-wind-turbines>
17. *Wind turbine recycling*. (2026, May 22). Energy.gov. <https://www.energy.gov/cmei/systems/wind-turbine-recycling>
18. *Economics*. (2026, August 13). Energy.gov. <https://www.energy.gov/cmei/systems/windexchange/economics>
19. Science Insights Team. (2026a, March 29). *How does wind energy affect the economy: Jobs & costs - ScienceInsights*. ScienceInsights. <https://scienceinsights.org/how-does-wind-energy-affect-the-economy-jobs-costs/>
20. Wanderley, J., & Levi, C. (2023). Numerical improvement to Glauert correction for the flow around a wind turbine. *The Aeronautical Journal*, 128(1320), 230–244. <https://doi.org/10.1017/aer.2023.54>
21. Shauri D (2026) Horizontal Axis Wind Turbine Design